



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2: Project Management

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Project Management



"Plans are nothing, planning is everything"

-Dwight D. Eisenhower

Software Project Management

❑ What is a project?

All projects are a temporary effort to create value through a unique product, service or result. All projects have a beginning and an end. They have a team, a budget, a schedule and a set of expectations the team needs to meet.

❑ What is project management?

Project management is the use of specific knowledge, skills, tools and techniques to deliver something of value to people.

❑ What is software project management?

Software project management is an art and science of planning and leading software projects. It is a sub-discipline of project management in which software projects are planned, implemented, monitored and controlled.

Software Project Management

A software project manager :

- ❑ Defines the requirements of the project,
- ❑ Builds the project team,
- ❑ Lays out a blue print for the whole project including the project scope and parameters,
- ❑ Clearly communicates the goals of the project to the team; the targets to be achieved,
- ❑ Allots budget to the various tasks to be completed, and
- ❑ Ensures that the expectations of the Board of Directors and Stakeholders are met through timely completion of the project.

Software Project Manager – Different roles

Leader

- A software project manager must lead his/her team towards success.
- He/She should provide them direction and make them understand what is expected of them.
- Clearly explain the roles of each member of the team.
- He/She must build a team comprising of individuals with different skills so that each member contributes effectively to the best of their abilities.

Liaison

- The project manager is a link between his/her clients, his/her team and his/her own supervisors.
- He/She must coordinate and transfer all the relevant information from the clients to his/her team and report to the upper management.
- He/She should work closely with analysts, software designers and other staff members and communicate the goals of the project.
- He/She monitors the progress of the project, taking action accordingly.

Software Project Manager – Different roles

Mentor

- He/She must be there to guide his/her team at every step and ensure that the team has cohesion.
- He/She provides advice to his/her team wherever they need it and points them in the right direction.

Fundamentals of Software Project Management

- Software Engineering is very relevant for large projects which involve
 - Several people
 - Several months
 - Planning and Controlling
 - Coordination
 - Efficient Project Management

- **Good project management cannot guarantee success, but poor management on significant projects always leads to failure.**

Quality of Projects

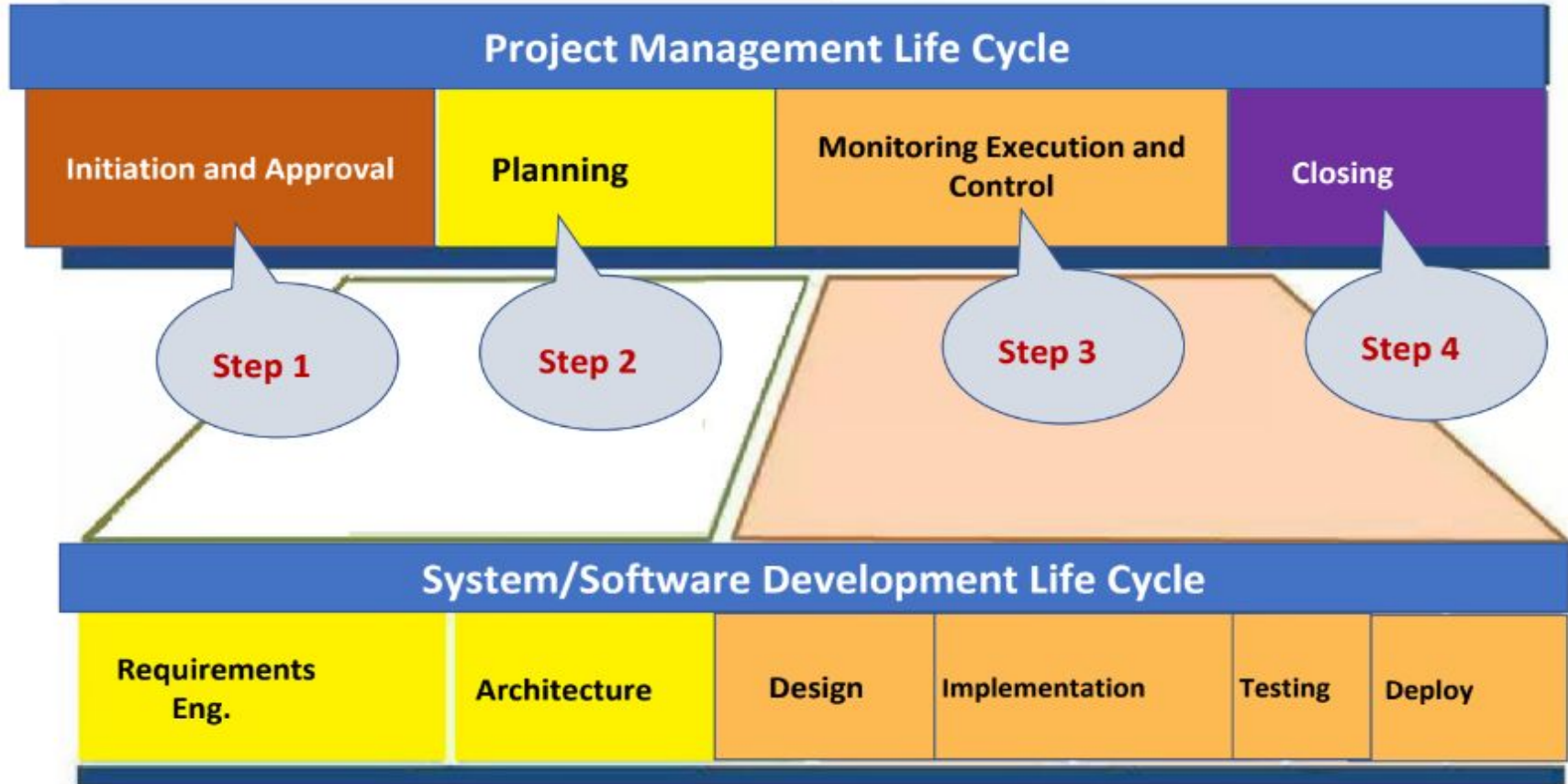
- Main objective: is to achieve project goals and targets while keeping in mind the project scope, time, quality, and cost.



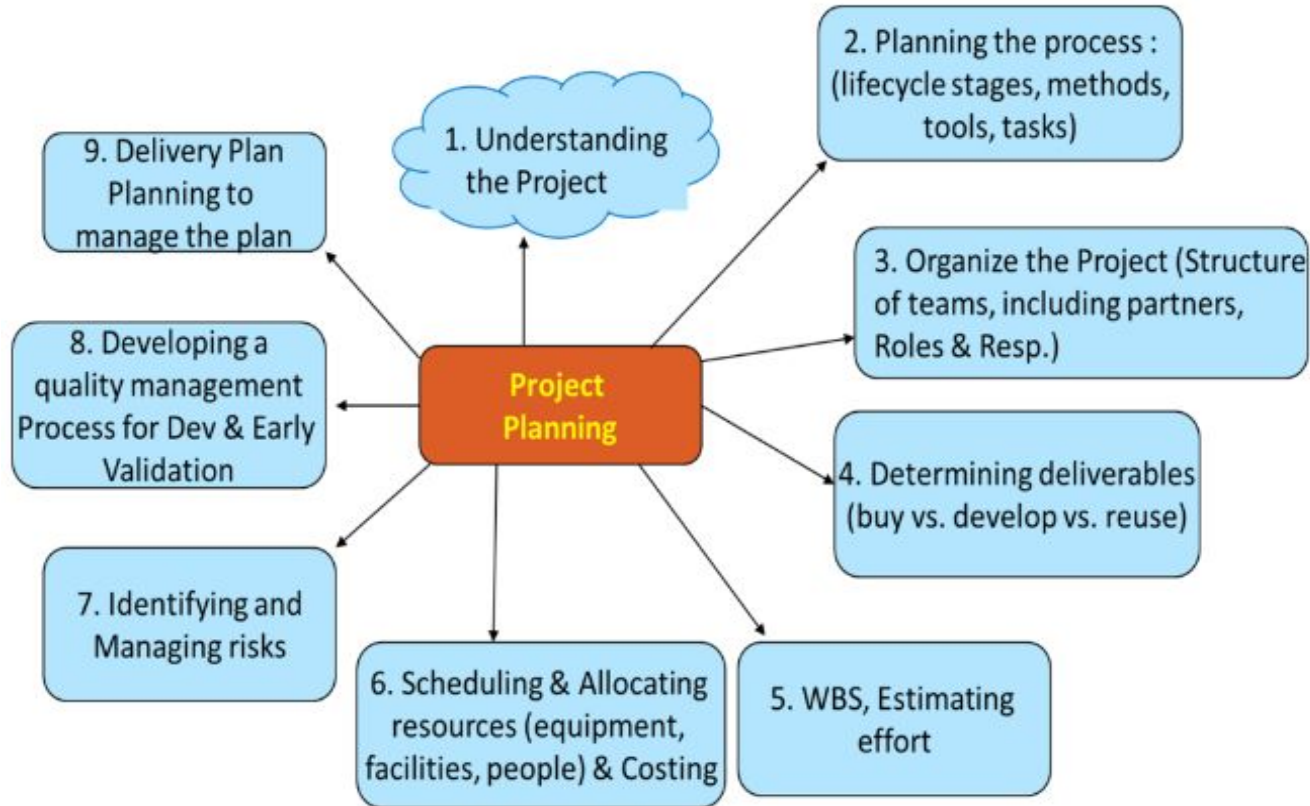
Quality of Projects

- The project management triangle visualizes the problem of “triple constraints”—the need to balance scope, cost, and time in order to maintain a high-quality final product.
- Every project manager who has known the frustration of trying to keep their team productive with too little time, not enough budget, and an impossibly large scope has experienced the project management triangle in action.
- No single variable of the project triangle can be changed without making tradeoffs with the other two points of the triangle.
- It's the project manager's job to balance all three elements in order to keep their project within budget and on deadline while still fulfilling the specifications of the project's scope.

Software Project Management lifecycle



Project Planning





Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2: Project Management

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Project Planning: Agile planning

Agile Planning

- Agile methods of software development are iterative approaches where the software is developed and delivered to customers in increments.
- Unlike plan-driven approaches, the functionality of these increments is not planned in advance but is decided during the development.
 - The decision on what to include in an increment depends on progress and on the customer's priorities.
- The customer's priorities and requirements change so it makes sense to have a flexible plan that can accommodate these changes.

Agile Planning

- Release planning, which looks ahead for several months and decides on the features that should be included in a release of a system.
- Iteration planning, which has a shorter-term outlook, and focuses on planning the next increment of a system. This is typically 2-4 weeks of work for the team.

Approaches to Agile Planning

Planning in Scrum

- Based on managing a project backlog (things to be done) with daily reviews of progress and problems
- The planning game
 - Developed originally as part of Extreme Programming (XP)
 - Dependent on user stories as a measure of progress in the project

Story based Planning

- The planning game is based on user stories that reflect the features that should be included in the system.
- The project team read and discuss the stories and rank them in order of the amount of time they think it will take to implement the story.
- Stories are assigned 'effort points' reflecting their size and difficulty of implementation
- The number of effort points implemented per day is measured giving an estimate of the team's 'velocity'
- This allows the total effort required to implement the system to be estimated

The planning game



Release and iteration planning

- Release planning involves selecting and refining the stories that will reflect the features to be implemented in a release of a system and the order in which the stories should be implemented.
- Stories to be implemented in each iteration are chosen, with the number of stories reflecting the time to deliver an iteration (usually 2 or 3 weeks).
- The team's velocity is used to guide the choice of stories so that they can be delivered within an iteration.

Task allocation

- During the task planning stage, the developers break down stories into development tasks.
 - A development task should take 4–16 hours.
 - All of the tasks that must be completed to implement all of the stories in that iteration are listed.
 - The individual developers then sign up for the specific tasks that they will implement.
- Benefits of this approach:
 - The whole team gets an overview of the tasks to be completed in an iteration.
 - Developers have a sense of ownership in these tasks and this is likely to motivate them to complete the task.

Software delivery

- A software increment is always delivered at the end of each project iteration.
- If the features to be included in the increment cannot be completed in the time allowed, the scope of the work is reduced.
- The delivery schedule is never extended.

Agile planning difficulties

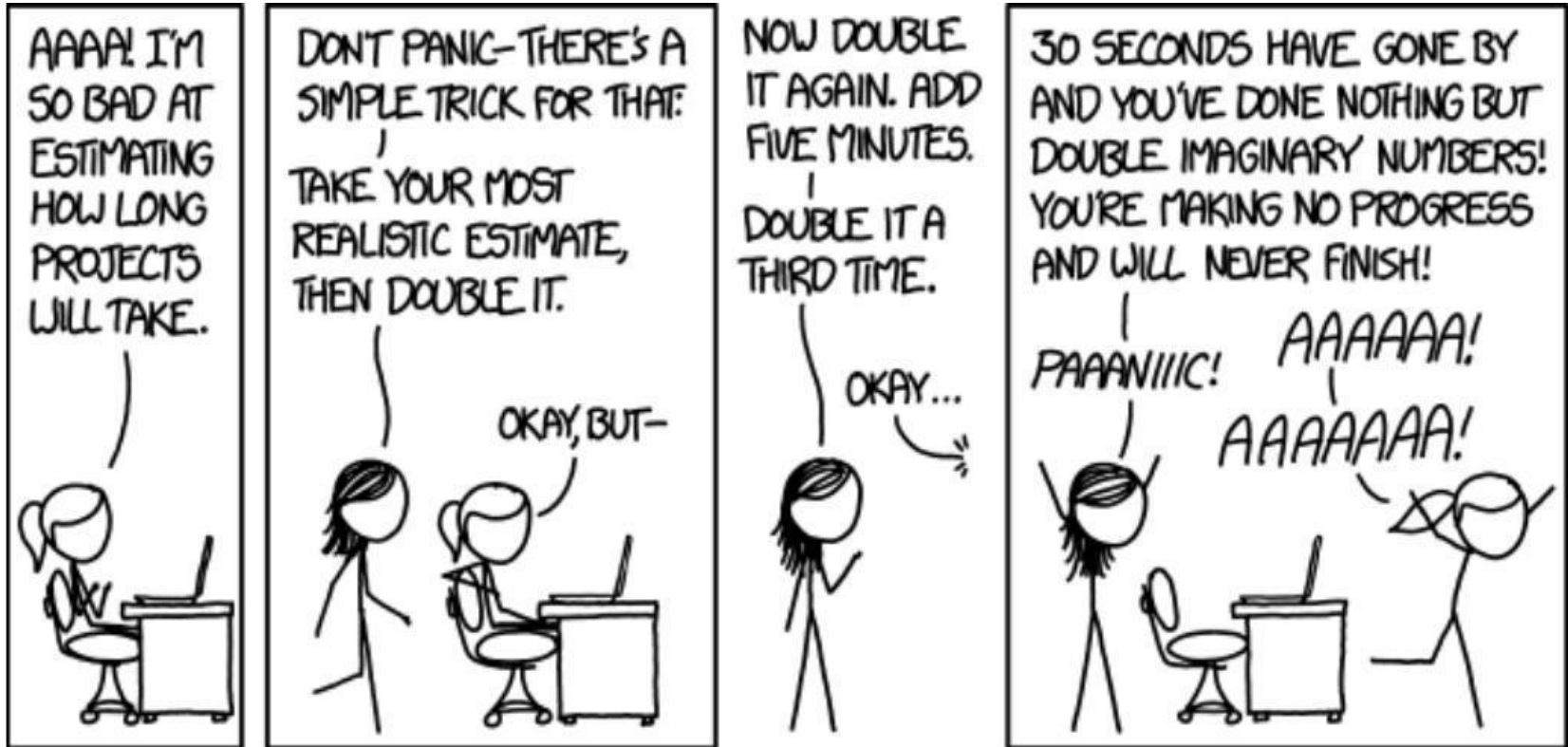
- Agile planning is reliant on customer involvement and availability.
- This can be difficult to arrange, as customer representatives sometimes have to prioritize other work and are not available for the planning game.
- Furthermore, some customers may be more familiar with traditional project plans and may find it difficult to engage in an agile planning process.

Agile planning applicability

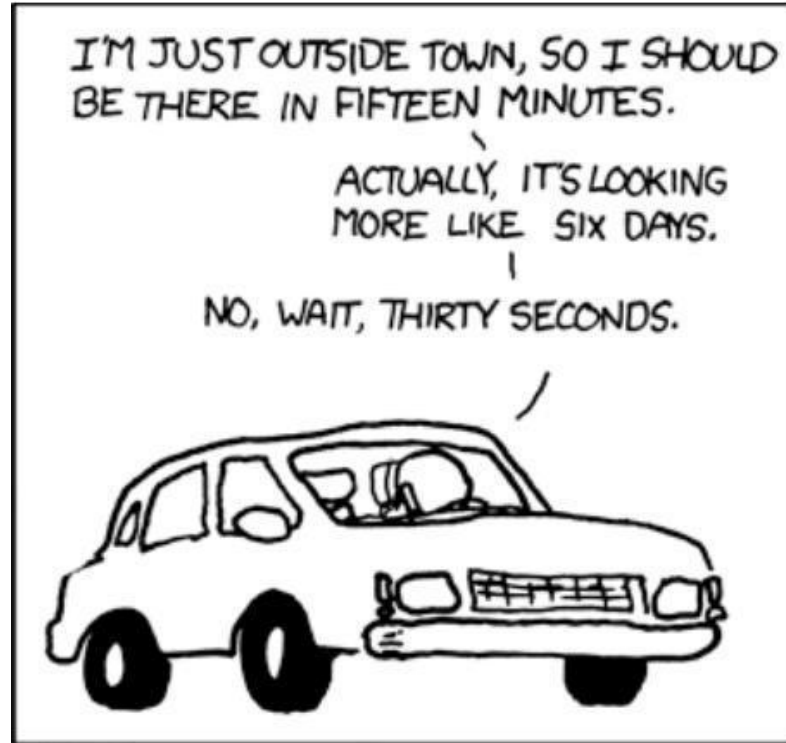
- Agile planning works well with small, stable development teams that can get together and discuss the stories to be implemented.
- However, where teams are large and/or geographically distributed, or when team membership changes frequently, it is practically impossible for everyone to be involved in the collaborative planning that is essential for agile project management.

Time Estimation

Time estimation



Time estimation



THE AUTHOR OF THE WINDOWS FILE
COPY DIALOG VISITS SOME FRIENDS.

A work breakdown structure (WBS) is a description of the work that will be done for a project

CoCoMo (Constructive Cost Model)

- It is a regression model based on LoC, i.e number of Lines of Code.
- It is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality.
- The key parameters which define the quality of any software products, which are also an outcome of the CoCoMo are primarily Effort & Schedule:
 - Effort: Amount of labor that will be required to complete a task. It is measured in person-months units.
 - Schedule: Simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in units of time such as weeks, months.

More about WBS – Estimation approaches

- Different models of CoCoMo have been proposed to predict the cost estimation at different levels, based on the amount of accuracy and correctness required.

- All of these models can be applied to a variety of projects, whose characteristics determine the value of constant to be used in subsequent calculations. These characteristics pertaining to different system types are mentioned below.
 - **Organic** – A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.
 - Example: An Inventory Management system
 - **Semi-detached** – A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, knowledge of the various programming environment lie in between that of organic and Embedded.
 - Example: Developing an OS, DBMS, etc

More about WBS – Estimation approaches

- ❑ **Embedded** – A software project requiring the highest level of complexity, creativity, and experience requirement fall under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models. Software for an ATM.

- ❑ There are 3 different types of CoCoMo models
 1. Basic
 2. Intermediate
 3. Detailed

More about WBS – Estimation approaches

Basic model

$$\text{Effort (E)} = a (\text{KLOC})^b$$

$$\text{time} = c (\text{Effort})^d$$

$$\text{Persons required} = \text{Effort} / \text{time}$$

The constant values a,b,c and d for the Basic Model for the different categories of system:

Software Project	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Project scheduling

- *Software project scheduling* is an activity that distributes estimated effort across the planned project duration by allocating the effort to specific software engineering tasks.
- It is important to note, however, that the schedule evolves over time.
- During early stages of project planning, a macroscopic schedule is developed.
This type of schedule identifies all major process framework activities and the product functions to which they are applied.
- As the project gets under way, each entry on the macroscopic schedule is refined into a detailed schedule.
- Here, specific software actions and tasks (required to accomplish an activity) are identified and scheduled.

Project scheduling

- Scheduling for software engineering projects can be viewed from two rather different perspectives.
 1. An end date for release of a computer-based system has already (and irrevocably) been established. The software organization is constrained to distribute effort within the prescribed time frame.
 2. The second view of software assumes that rough chronological bounds have been discussed but that the end date is set by the software engineering organization. Effort is distributed to make best use of resources, and an end date is defined after careful analysis of the software.
- Unfortunately, the first situation is encountered far more frequently than the second.

Worked example

A software company is developing a **Campus Event Management System** for a university. The development team estimates the system size to be **10 KLOC**. Since it is a relatively familiar application, the project is classified as **Organic mode**. Using the COCOMO model, estimate the **effort, development time, and number of people required**

Worked example - Solution

Step 1: Effort

$$E = 2.4(10)^{1.05}$$

$$E \approx 26.93 \text{ person-months}$$

Step 2: Development time

$$T = 2.5(26.93)^{0.38}$$

$$T \approx 8.77 \text{ months}$$

Step 3: People required

$$P = \frac{26.93}{8.77}$$

$$P \approx 3.07$$

So the estimate is approximately:

27 person-months of effort, 8.8 months of development time, and an average team size of about 3 people.

Project scheduling

Basic principles

1. Compartmentalization
2. Interdependency
3. Time allocation
4. Effort validation
5. Defined responsibilities
6. Defined outcomes
7. Defined milestones

Risk Management

Risk management

- Risk management is concerned with identifying risks and drawing up plans to minimise their effect on a project.
- Software risk management is important because of the inherent uncertainties in software development.
 - These uncertainties stem from loosely defined requirements, requirements changes due to changes in customer needs, difficulties in estimating the time and resources required for software development, and differences in individual skills.
- You have to anticipate risks, understand the impact of these risks on the project, the product and the business, and take steps to avoid these risks.

Risk classification

- There are two dimensions of risk classification
 - The type of risk (technical, organizational, ..)
 - What is affected by the risk:
 - **Project risks** affect schedule or resources;
 - **Product risks** affect the quality or performance of the software being developed;
 - **Business risks** affect the organisation developing or procuring the software.

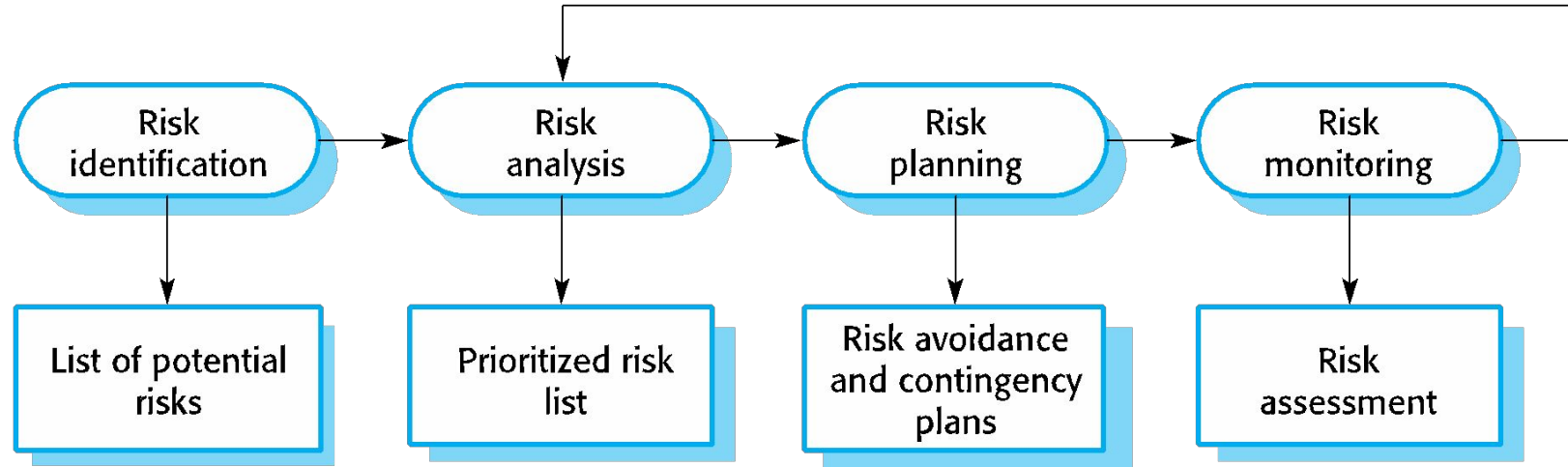
Examples of project, product, and business risks

Risk	Affects	Description
Staff turnover	Project	Experienced staff will leave the project before it is finished.
Management change	Project	There will be a change of organizational management with different priorities.
Hardware unavailability	Project	Hardware that is essential for the project will not be delivered on schedule.
Requirements change	Project and product	There will be a larger number of changes to the requirements than anticipated.
Specification delays	Project and product	Specifications of essential interfaces are not available on schedule.
Size underestimate	Project and product	The size of the system has been underestimated.
CASE tool underperformance	Product	CASE tools, which support the project, do not perform as anticipated.
Technology change	Business	The underlying technology on which the system is built is superseded by new technology.
Product competition	Business	A competitive product is marketed before the system is completed.

The Risk Management process

- Risk identification
 - Identify project, product and business risks;
- Risk analysis
 - Assess the likelihood and consequences of these risks;
- Risk planning
 - Draw up plans to avoid or minimise the effects of the risk;
- Risk monitoring
 - Monitor the risks throughout the project;

The Risk Management process



Risk Indicators

Risk type	Potential indicators
Estimation	Failure to meet agreed schedule; failure to clear reported defects.
Organizational	Organizational gossip; lack of action by senior management.
People	Poor staff morale; poor relationships amongst team members; high staff turnover.
Requirements	Many requirements change requests; customer complaints.
Technology	Late delivery of hardware or support software; many reported technology problems.
Tools	Reluctance by team members to use tools; complaints about CASE tools; demands for higher-powered workstations.

Quality management

□ What is it?

You know quality when you see it, and yet, it can be an elusive thing to define. But for computer software, quality is something that we must define, and that's what we'll do now.

□ Who does it?

Everyone—software engineers, managers, all stakeholders—involved in the software process is responsible for quality.

□ Why is it important?

You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

RACI matrix

- A RACI chart, also called a RACI matrix, is a type of responsibility assignment matrix (RAM) in project management. In practice, it's a simple spreadsheet or table that lists all stakeholders on a project and their level involvement in each task, denoted with the letters R, A, C or I.

- Once these roles are defined, assignments can be attributed to the roles and work can begin.

- **R, A, C, I** stands for:
 1. **Responsible**
 2. **Accountable**
 3. **Consulted**
 4. **Informed**

RACI matrix (Summary)

- ❑ **Responsible** is “The Doer” the individual or Individuals who perform an activity — responsible for an action, there may be several Rs.
- ❑ **Accountable** “The Buck Stops Here” The individual who is ultimately Accountable has the yes/no and power to veto an activity or decision providing the Sponsor is in agreement. Only one “A” can be assigned to an activity or task as you can't have 2 people driving the bus.
- ❑ **Consulted** “In the Loop” The individual or individuals who need to be consulted prior to a task or action being completed their expert knowledge relating to the task is considered important. This is a two-way communication.
- ❑ **Informed** The individual or individuals who need(s) to be informed after a decision or action is taken. One-way communication.

Outcomes of project planning

What are the outcomes of Project Planning?

- Project Plan
- Work Breakdown Structure
- Schedule
- Resource Management
- Plans for communication, metrics, etc
- Risk management plan
- Different roles

Contents of Project plan

- Introduction
- Deliverables of the project
- Process model
- Organization of project
- Standards, guidelines, procedures
- Management activities
- Risks
- Staffing
- Methods and techniques
- Quality criteria/assurance
- Work packages
- Resources
- Budget and schedule
- Change control process
- Delivery means

A good template is [here](#).

Project monitoring and control

What is Project Monitoring and Control?

- Project Monitoring and Control includes those processes performed to observe project execution so that potential problems can be identified in a timely manner and corrective action can be taken, when necessary, to control the execution of the project.
- Project control activities involve making decisions or adjustments in dimensions like time, cost, organizational structure, scope etc. for the project under execution.

Project monitoring and control

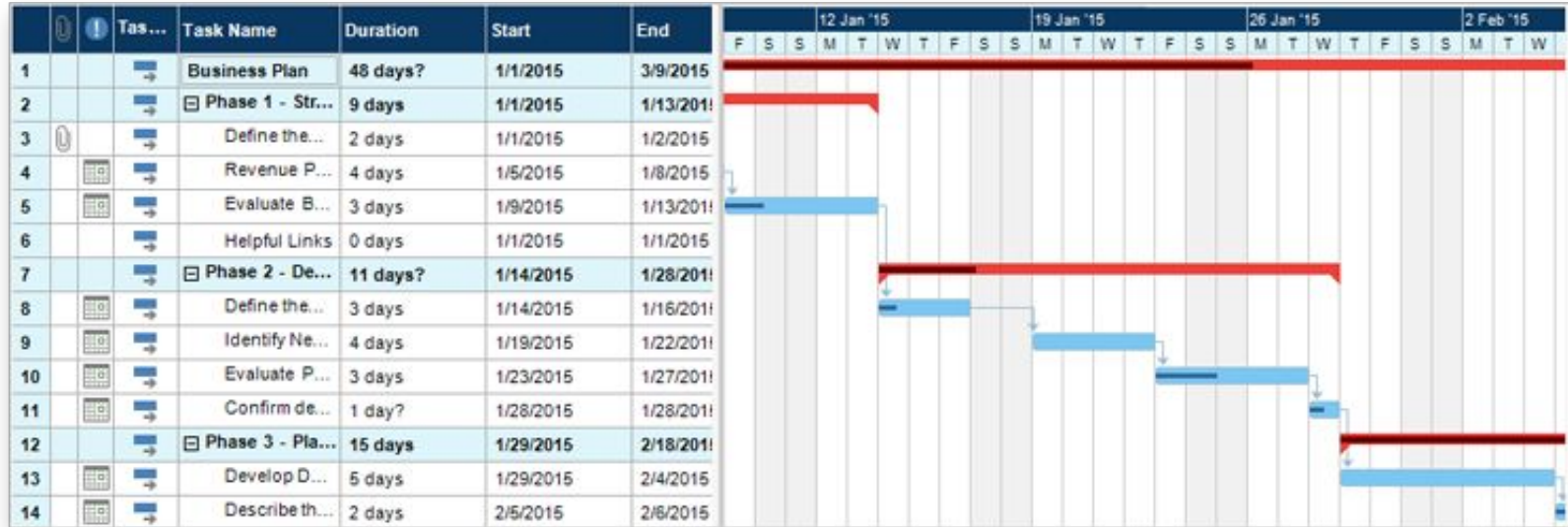
Project monitoring and control process could include:

- Monitoring and controlling project work
 - Collects, measures and disseminates performance information, assess trends to identify potential items requiring corrective action or monitoring for risks.
 - This can result in corrective or preventive actions and request for changes
 - Periodically do **Critical Path Analysis**
- Ensuring that all of the change controls are being meticulously followed
- Ensuring that all of the scope, deliverables, documents are periodically updated with the project plans.
- Controlling the Quality triangle
- Managing the project team, performance management and communication management

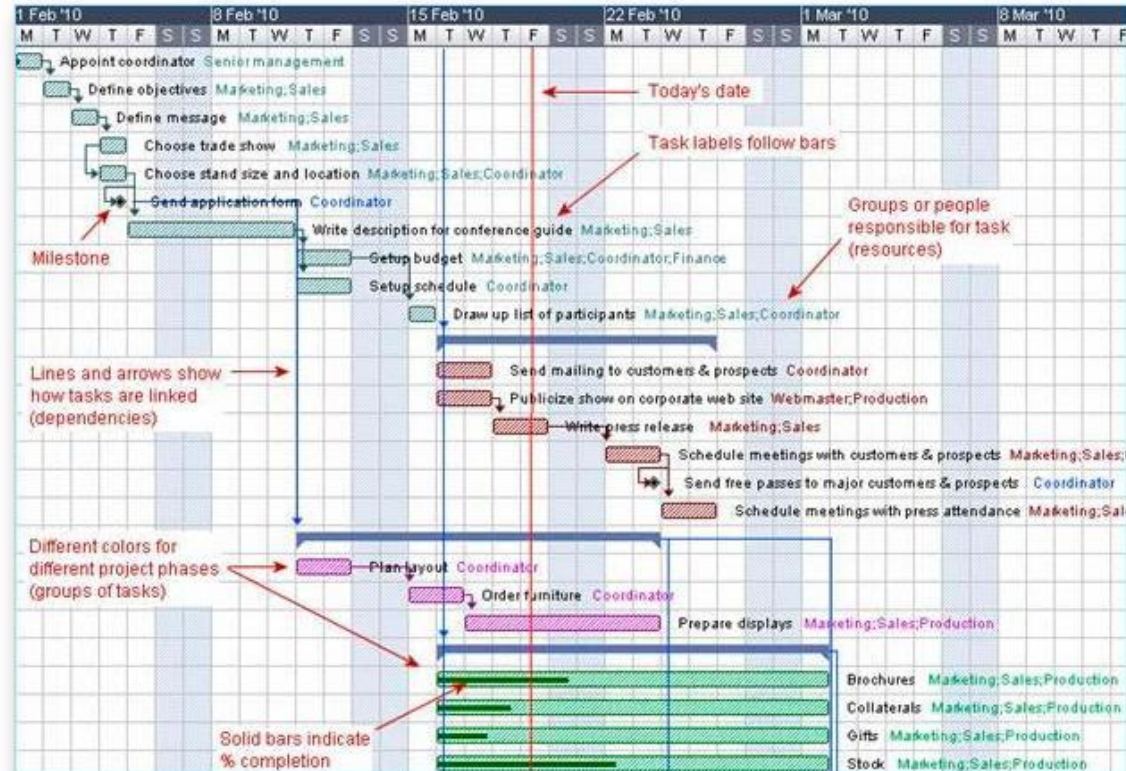
Gantt Chart

- A Gantt chart, commonly used in project management, is one of the most popular and useful ways of showing activities (tasks or events) displayed against time. On the left of the chart is a list of the activities and along the top is a suitable time scale. Each activity is represented by a bar; the position and length of the bar reflects the start date, duration and end date of the activity. This allows you to see at a glance:
 - What the various activities are
 - When each activity begins and ends
 - How long each activity is scheduled to last
 - Where activities overlap with other activities, and by how much
 - The start and end date of the whole project
- To summarize, a Gantt chart shows you what has to be done (the activities) and when (the schedule).

Gantt Chart



Gantt Chart

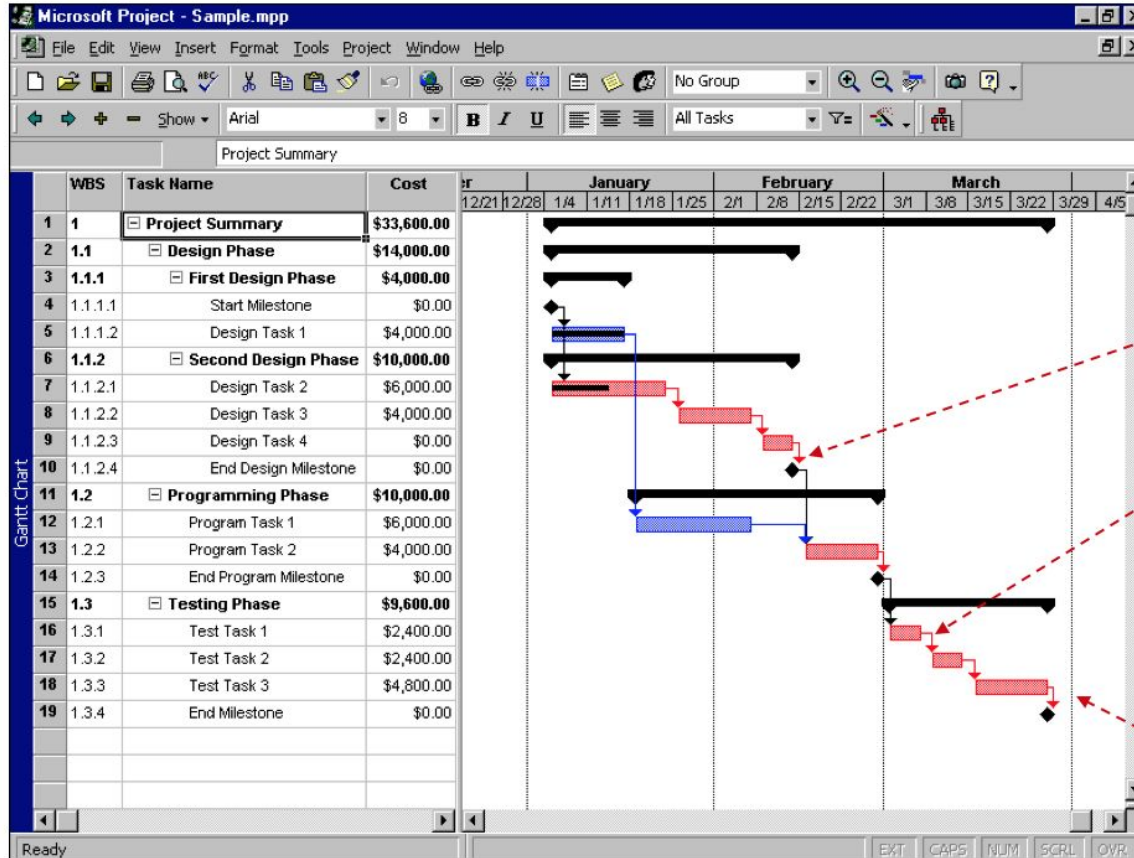


Gantt Chart

What Is a Critical Path in a Gantt Chart?

- A *critical path* in a Gantt chart refers to the phenomenon in which any individual task causes a delay in the related sequence of tasks, thus pushing back a project's overall end date.
- In project management, the “critical path” refers to the **longest sequence of dependent or floating tasks** that must be completed in order to get the project done on time.
- Activities that have a total slack time of zero are considered to be on the critical path.
- One of the most commonly used tools for project management is Jira. This software is used for **bug tracking, issue tracking and project management**.

Gantt Chart



• Milestone

• Dependencies relationships between activities are added

• Red bars represents critical activities

Project Management: Teamwork

- Most software engineering is a group activity
 - The development schedule for most non-trivial software projects is such that they cannot be completed by one person working alone.
- A good group is cohesive and has a team spirit. The people involved are motivated by the success of the group as well as by their own personal goals.
- Group interaction is a key determinant of group performance.
- Flexibility in group composition is limited
- Managers must do the best they can with available people.

Group cohesiveness

- In a cohesive group, members consider the group to be more important than any individual in it.
- The advantages of a cohesive group are:
 - Group quality standards can be developed by the group members.
 - Team members learn from each other and get to know each other's work; Inhibitions caused by ignorance are reduced.
 - Knowledge is shared. Continuity can be maintained if a group member leaves.
 - Refactoring and continual improvement is encouraged. Group members work collectively to deliver high quality results and fix problems, irrespective of the individuals who originally created the design or program.

Team spirit

- Alice, an experienced project manager, understands the importance of creating a cohesive group. As they are developing a new product, she takes the opportunity of involving all group members in the product specification and design by getting them to discuss possible technology with elderly members of their families. She also encourages them to bring these family members to meet other members of the development group.
- Alice also arranges monthly lunches for everyone in the group. These lunches are an opportunity for all team members to meet informally, talk around issues of concern, and get to know each other. At the lunch, Alice tells the group what she knows about organizational news, policies, strategies, and so forth. Each team member then briefly summarizes what they have been doing and the group discusses a general topic, such as new product ideas from elderly relatives.
- Every few months, Alice organizes an 'away day' for the group where the team spends two days on 'technology updating'. Each team member prepares an update on a relevant technology and presents it to the group. This is an off-site meeting in a good hotel and plenty of time is scheduled for discussion and social interaction.

The effectiveness of a team

□ The people in the group

- You need a mix of people in a project group as software development involves diverse activities such as negotiating with clients, programming, testing and documentation.

□ The group organization

- A group should be organized so that individuals can contribute to the best of their abilities and tasks can be completed as expected.

□ Technical and managerial communications

- Good communications between group members, and between the software engineering team and other project stakeholders, is essential.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2 - Lecture 2-3

Compiled by
Rajesh Banginwar

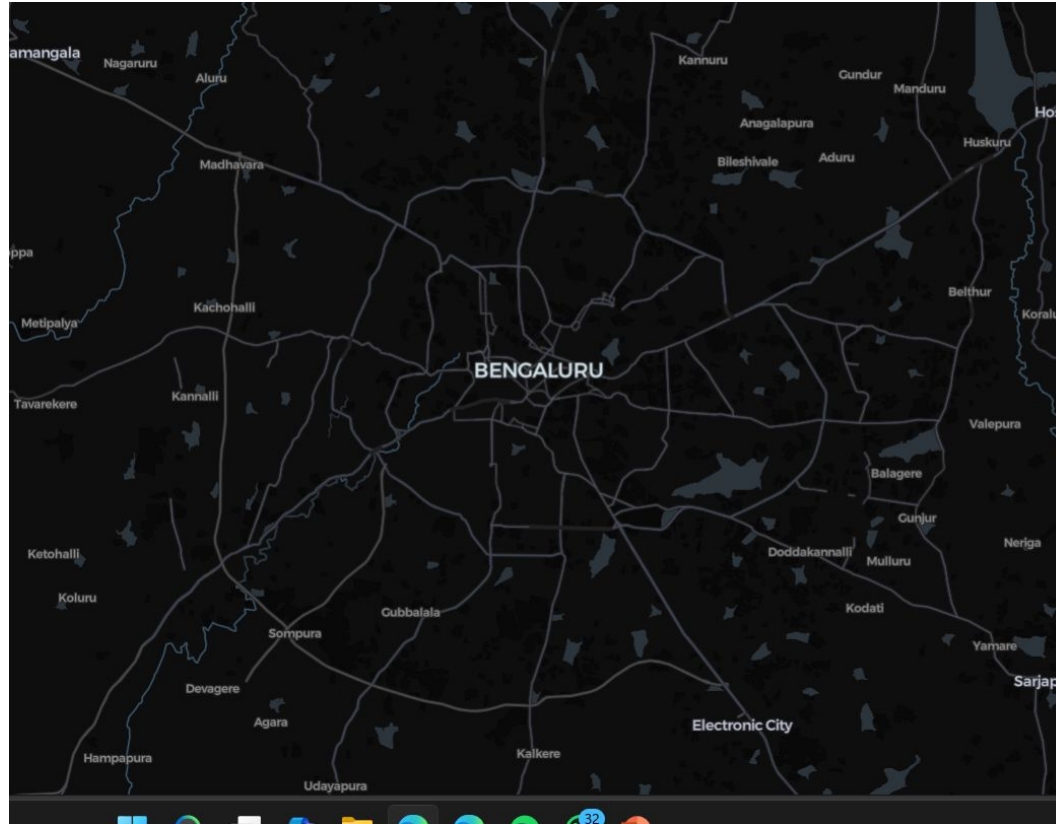
Department of Computer Science and Engineering

Software Architecture

- Views and Abstraction
- Case Study: Therac-25
- Software Architecture
 - Definitions, Importance
 - Software Design vs. Software Architecture
- Architecting software
 - Integrating Architectural Decisions into the SW Development Process
 - Common Software Architectures
 - Documentation

- Views and Abstraction
- Case Study: Therac-25
- Software Architecture
 - Definitions, Importance
 - Software Design vs. Software Architecture
- Architecting software
 - Integrating Architectural Decisions into the SW Development Process
 - Common Software Architectures
 - Documentation

Namma Bengaluru



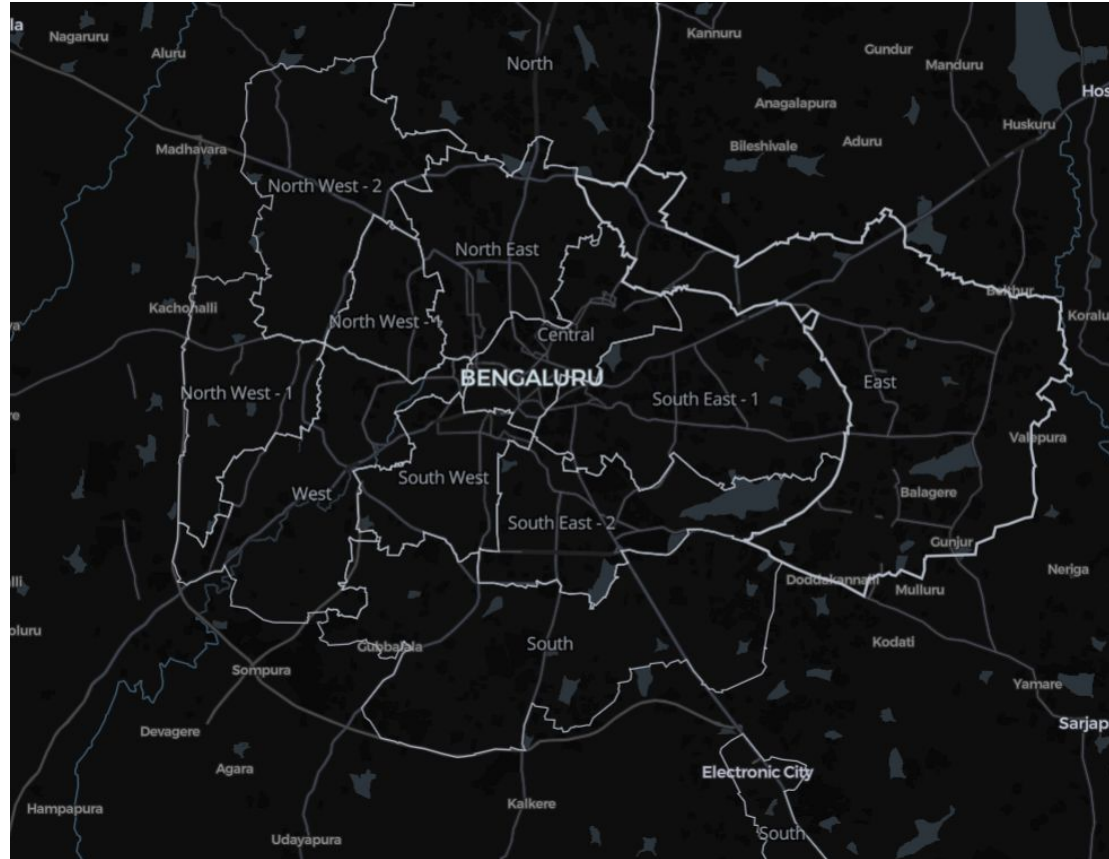




Bengaluru Map but with BESCOM Divisions



Bengaluru Map but with BWSSB Divisions



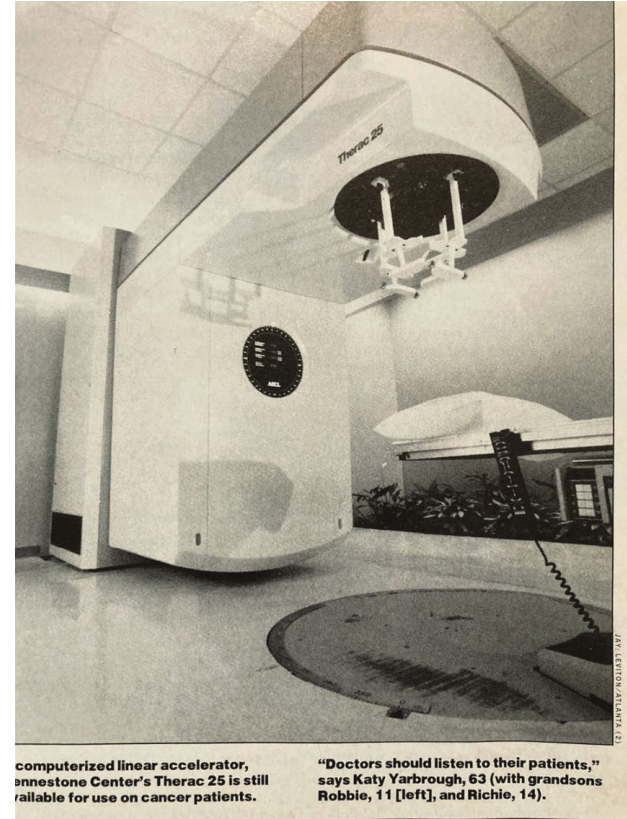
Bengaluru Map with state assembly demarcations



Abstracted Views

- Abstracted views focus on conveying specific information
- They have a well-defined purpose
- Shows only necessary information
- Abstract away unnecessary details
- Use legends/annotations to remove ambiguity
- Multiple views of the same object tell a larger story

- Views and Abstraction
- **Case Study: Therac-25**
- Software Architecture
 - Definitions, Importance
 - Software Design vs. Software Architecture
- Architecting software
 - Integrating Architectural Decisions into the SW Development Process
 - Common Software Architectures
 - Documentation

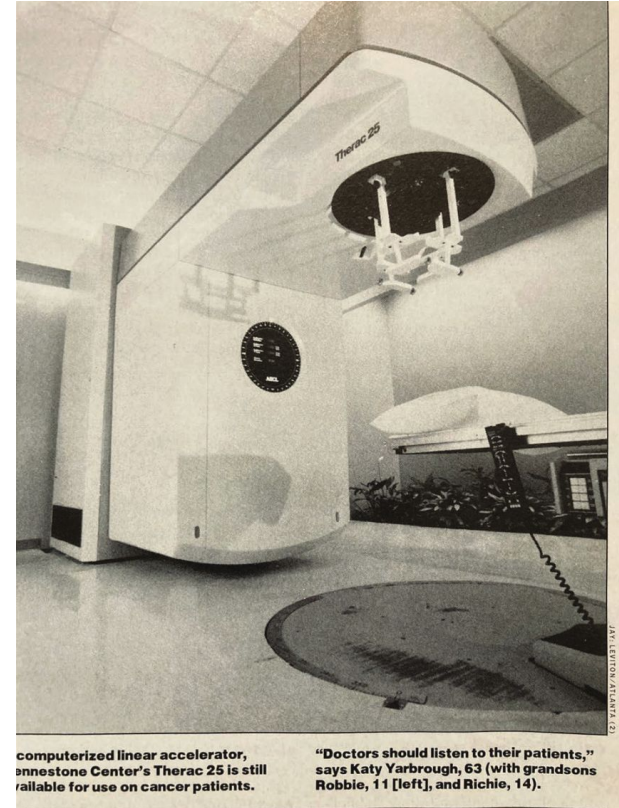


System Overview – What is Therac-25?

- ❑ **Therac-25:** Computer-controlled radiation therapy machine.
- ❑ **Purpose:** Deliver controlled dose's of radiation to cancer patients.
- ❑ **Design Features:**
 - Software replaced most hardware-based safety mechanisms.
 - Multiple treatment modes (low-energy X-ray, high-energy electron beam).
 - Operator used a terminal to input treatment settings.
- ❑ **Legacy Reuse:** Code adapted from Therac-6 and Therac-20, which had physical safety interlocks.
- ❑ **Embedded System:** Real-time interaction between software and custom hardware.
- ❑ **Why this case?**
 - Embedded system with life-critical consequences.
 - Real-world failures due to poor software architecture.
 - Lessons applicable to medical, automotive, IoT, and industrial systems.
- ❑ **Era:** 1980s, but still cited in safety and architecture literature.

What Happened?

- ❑ Machine delivered massive overdoses of radiation.
- ❑ At least 6 incidents, multiple injuries and deaths.
- ❑ Software reused from older models, but hardware interlocks removed.
- ❑ System failed silently or with cryptic error messages.



Architectural Failures

- ❑ **Tight Coupling:** UI, control logic, and machine control shared global memory.
- ❑ **Race Conditions:** Unsafe state transitions when operators acted quickly.
- ❑ **Reused Legacy Code:** Assumed hardware interlocks would catch errors.
- ❑ **Lack of Fault Isolation:** No module validation or error containment.
- ❑ **Poor Observability:** No logging or diagnostics.

Architecture Principles Violated

- ❑ No separation of concerns.
- ❑ Absence of fault-tolerant or fail-safe layers.
- ❑ Reuse without architectural redesign.
- ❑ Ignored safety-critical coding and testing standards.
- ❑ Lack of defensive programming and redundancy.

Impact Assessment

- ❑ **Human Impact:** Patients injured or killed by overdoses.
- ❑ **Reputation Damage:** AECL faced lawsuits, global scrutiny.
- ❑ **Industry Impact:** Contributed to development of IEC 62304 and other medical safety standards.
- ❑ **Learning:** Software failures in physical systems can be lethal.

What Should Have Been Done?

- ❑ Introduce modular, layered architecture with fault isolation.
- ❑ Build-in software interlocks and state machine guards.
- ❑ Defensive programming techniques and fail-safe defaults.
- ❑ Thorough validation and verification.
- ❑ Include redundancy in both logic and controls.

Class Discussion Questions

- ☐ How can architecture enforce safety in embedded systems?
- ☐ How to prevent race conditions?
- ☐ Should safety checks be in software, hardware, or both?
- ☐ How do software architecture decisions relate to ethical responsibility?

Software Architecture: Abstraction

- Elements:
 - roles, responsibilities, behaviors, properties.
- Relationships between elements.
- Relationships to non-software elements :
 - Hardware, external systems
- Described from many different perspectives (views)

What is Software Architecture?

It is the top-level decomposition of the system being developed, into major components together with a characterization of how these components interact

- Software architecture of a computing system can be
 - used for communication among stakeholders
 - a blueprint for the system and project under development
 - used for negotiations and
 - balancing of functional and quality goals which happen during architecture

Software Architecture: Motivation

- Facilitates internal and external communication
- Describes design decisions and prescribes implementation Constraints.
- Relates to organizational structure
- Permits/precludes achieving non-functional requirements
- Control complexity
- Reason about and manage change
- Good basis for effort estimation



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

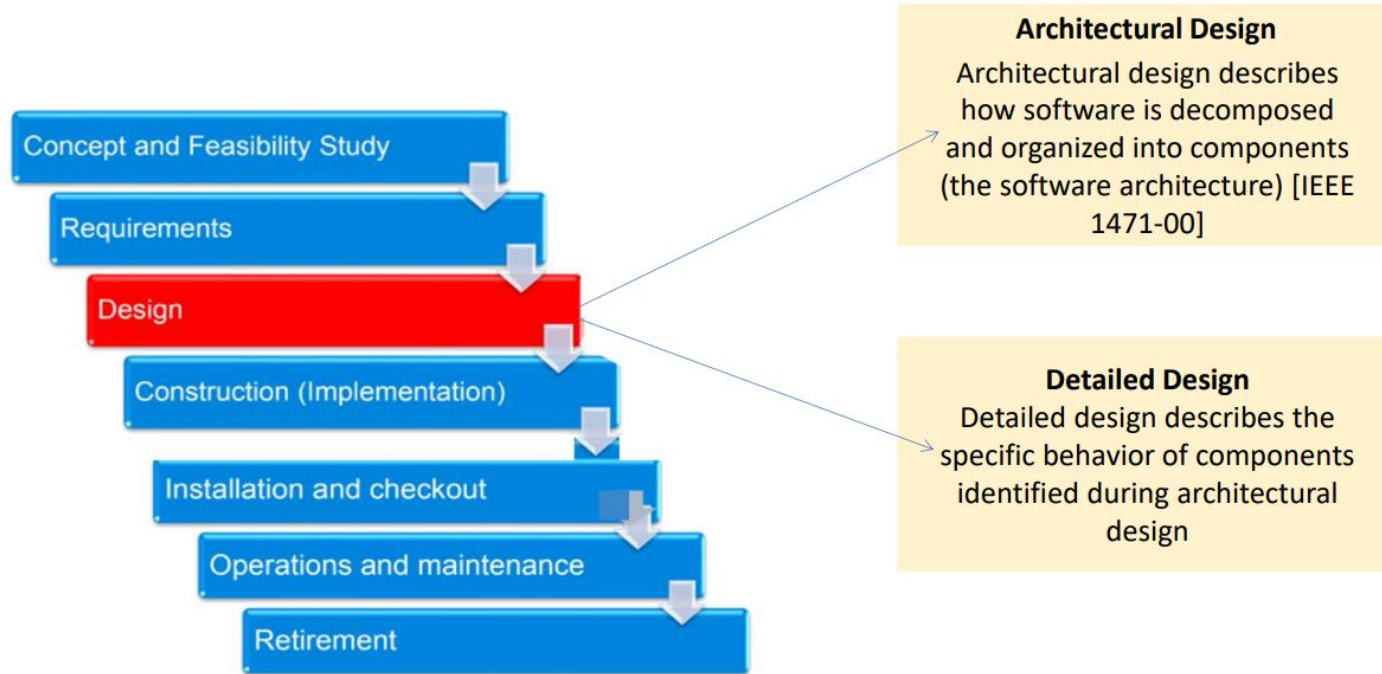
Unit 2: Project Management

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Software Design vs. Architecture

Context of Software Design



Levels of Abstraction

- Requirements
 - High-level “what” needs to be done

- Architecture (High-level Design)
 - High-level “how”, mid-level “what”

- Design (Low-level Design, e.g : Design Pattern)
 - Mid-level “how”, low-level “what”

- Code
 - Low-level “how”

Design v/s Architecture

Design Questions	Architectural Questions
How do I add a menu item in the app?	How do I extend app with a plugin ?
How can I make it easy to use a function?	What threads exist and how do they co-ordinate?
What lock protects this data?	How does Google scale to billions of hits per day ?
How does Google Rank pages?	Where should I put my firewalls ?
What encoder should I use for secure communication ?	What is the interface between subsystem ?

- Views and Abstraction
- Case Study: Therac-25
- Software Architecture
 - Definitions, Importance
 - Software Design vs. Software Architecture
- **Architecting software**
 - Integrating Architectural Decisions into the SW Development Process
 - Common Software Architectures
 - Documentation





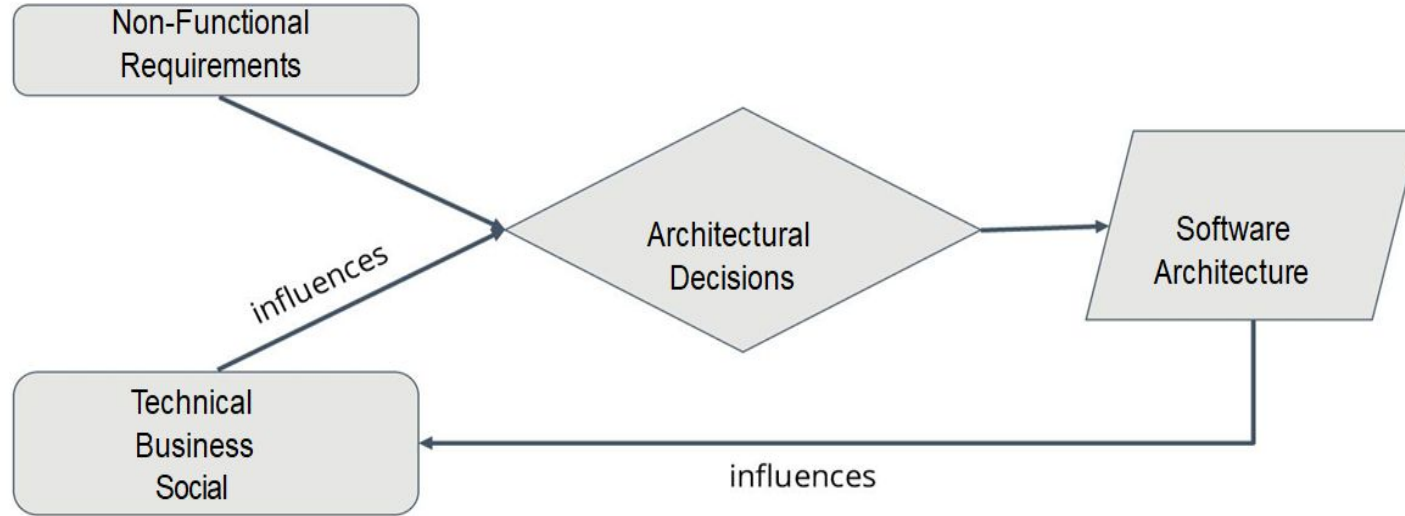
Every software system has an architecture

- Whether you know it or not
- Whether you like it or not
- Whether it's documented or not

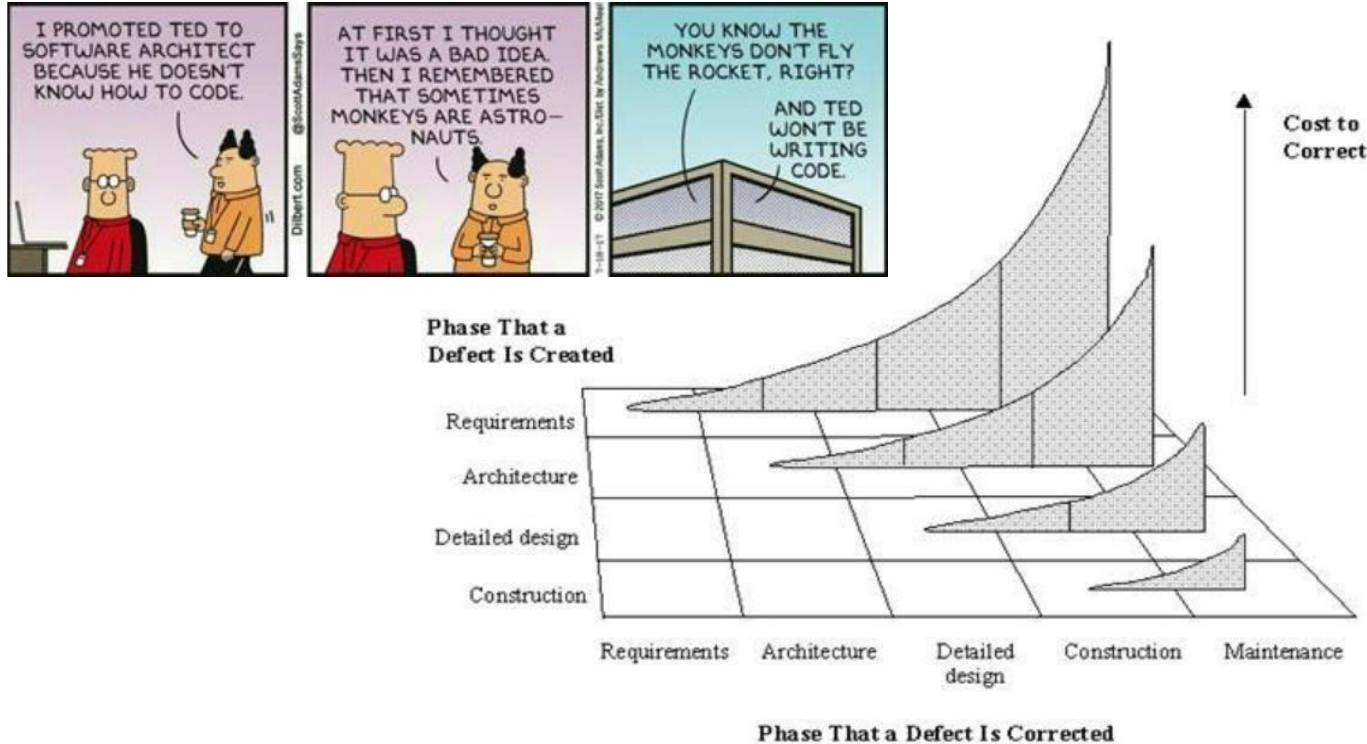
If you don't consciously elaborate the architecture, it will evolve by itself!



Every software system has an architecture



Every software system has an architecture



Copyright 1998 Steven C. McConnell. Reprinted with permission from *Software Project Survival Guide* (Microsoft Press, 1998).

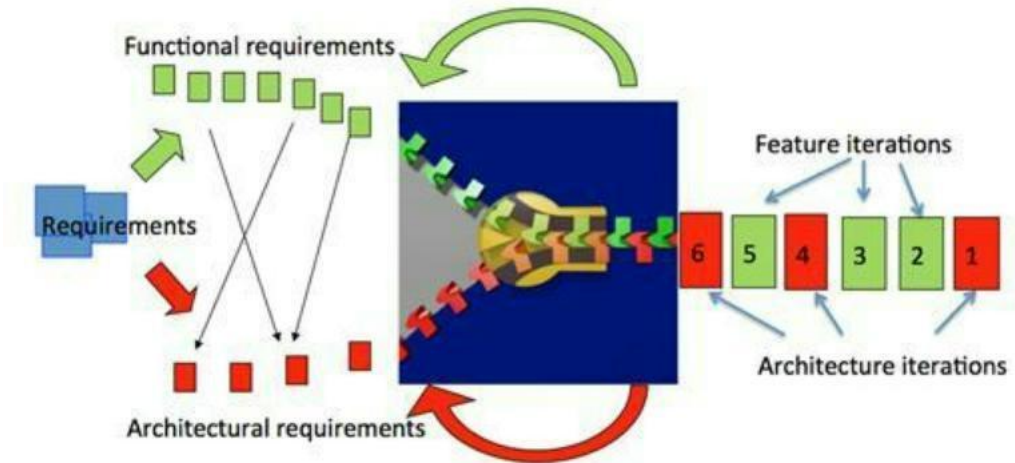
Agile and Architecture



The Zipper Model

How to Agilely Architect an Agile Architecture

by Stephany Bellomo, Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya



Common Software Architectural Patterns

Architectural Patterns

What is an Architectural pattern?

- An **architectural pattern** is a general, reusable solution to a commonly occurring problem in software architecture within a given context.
- Following traditional building architecture, a 'software architectural style' is a specific method of construction, characterized by the features that make it notable.
- An architectural style defines: a family of systems in terms of a pattern of structural organization; a vocabulary of components and connectors, with constraints on how they can be combined.

Architectural Patterns

An architectural style is a named collection of architectural design decisions

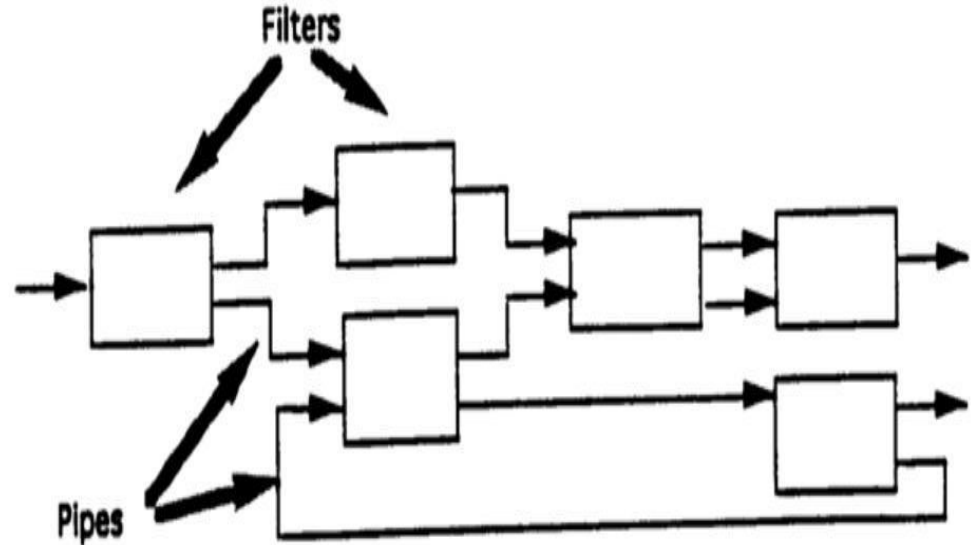
- that are applicable in a given development context,
- constrain architectural design decisions that are specific to a particular system within that context, and
- elicit beneficial qualities in each resulting system.

Some treat architectural patterns and architectural styles as the same, some treat styles as specializations of patterns. What they have in common is both patterns and styles are idioms for architects to use, they "provide a common language" or "vocabulary" with which to describe classes of systems.

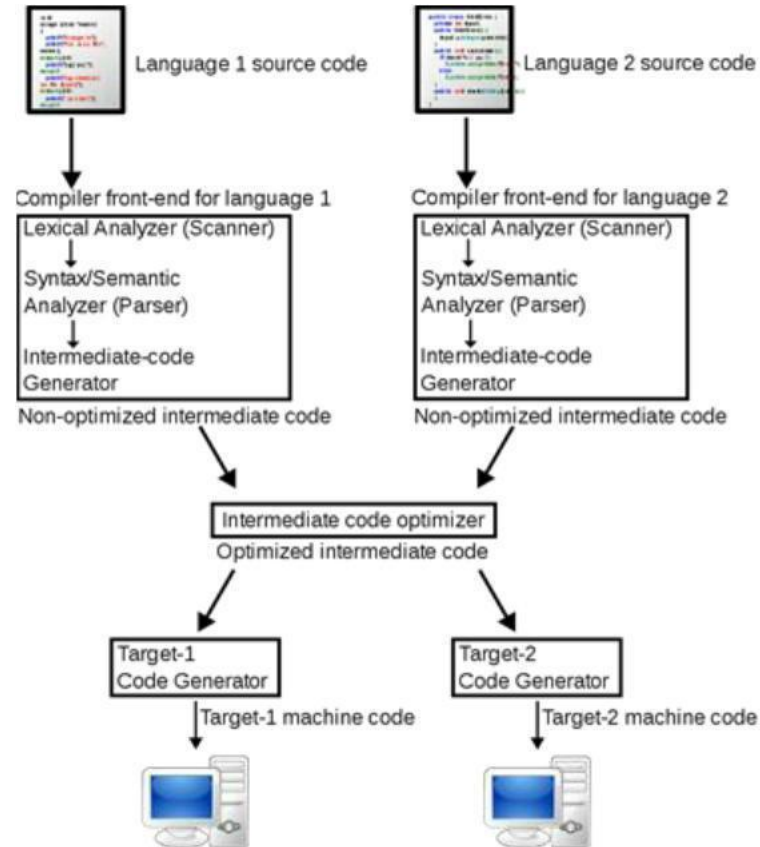
The main difference is that a pattern can be seen as a solution to a problem, while a style is more general and does not require a problem to solve for its appearance.

Pipes and Filters Architecture

- ❑ Data flows through a sequence of processing components (filters), connected by pipes.
- ❑ Each filter transforms the input data and passes it to the next.
- ❑ Pipes must compute local transformations
- ❑ Filters must not share state with other filters
- ❑ There must be no cycles

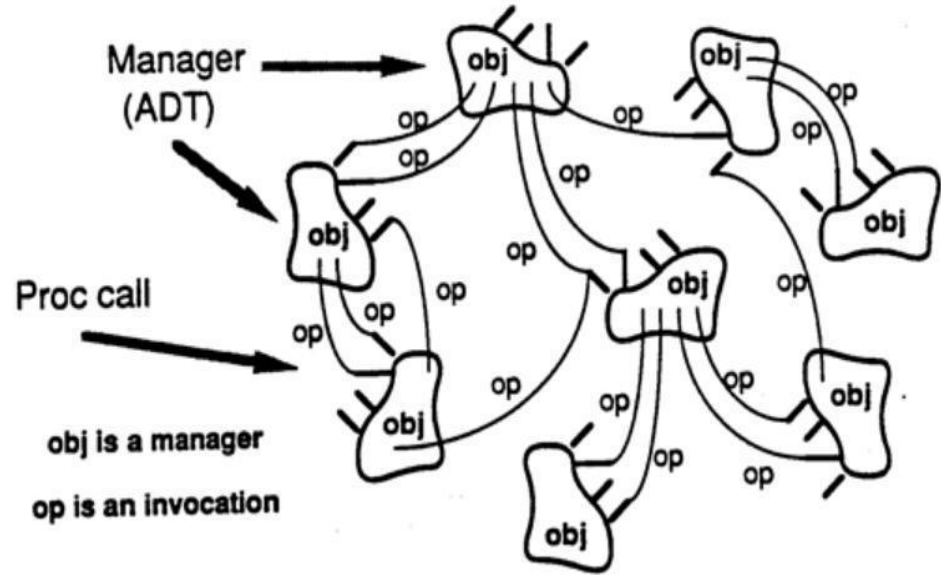


Example: Compilers

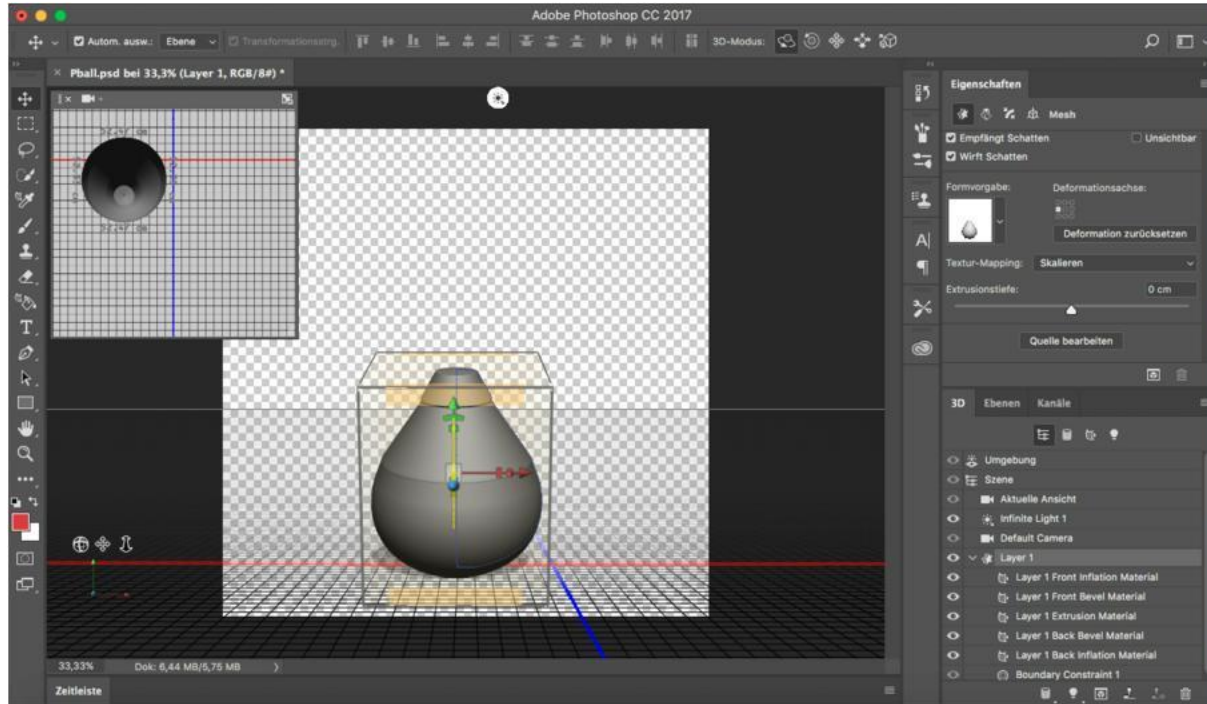


Object-Oriented Organization

- ❑ Organizes system as a collection of interacting objects, each with state and behavior.
- ❑ Promotes encapsulation, inheritance, and polymorphism.
- ❑ Great for systems with complex data models.
- ❑ Encourages code reuse and maintainability.

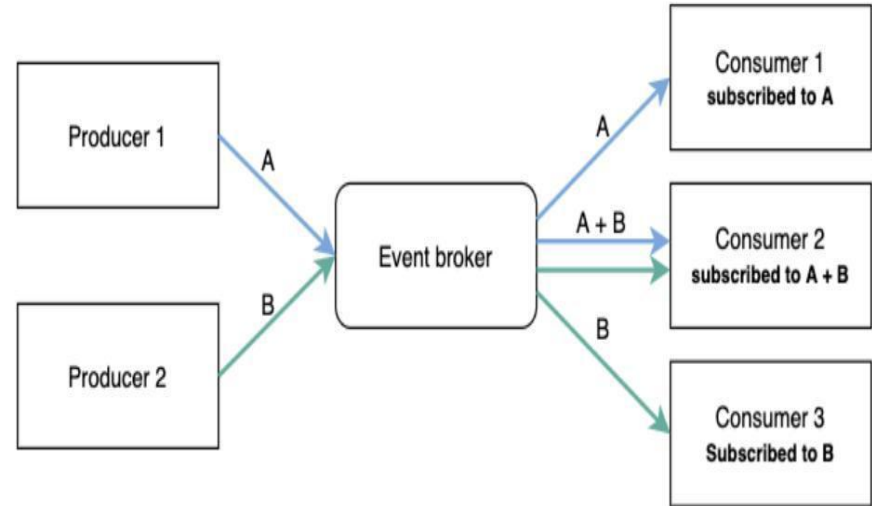


Example: Adobe Photoshop



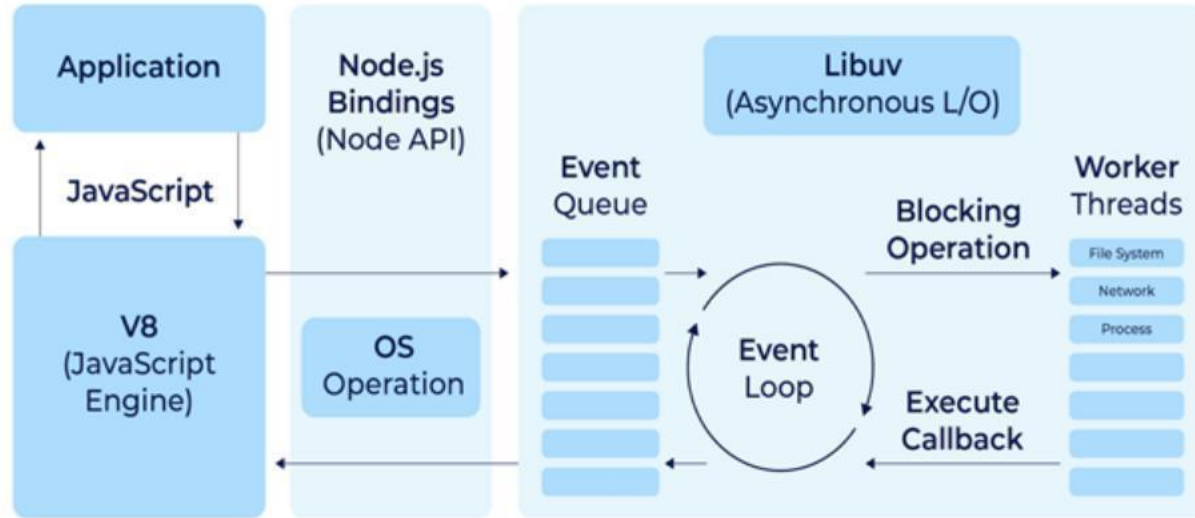
Event-Driven Architecture

- Components communicate through events (e.g., publish/subscribe).
- Suitable for asynchronous and reactive systems.
- Promotes loose coupling between components.



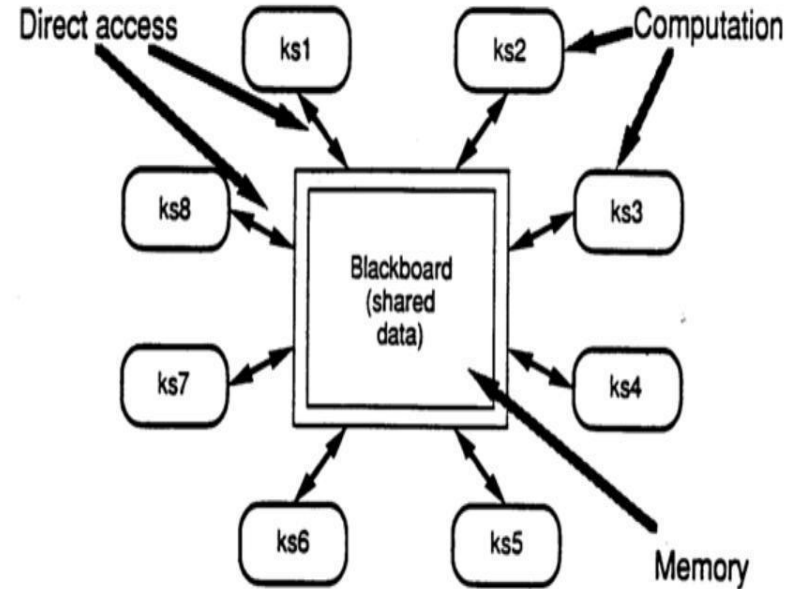
Example: Node.js

Node.js Architecture



Blackboard Architecture

- ❑ Multiple subsystems interact via a common knowledge base (blackboard).
- ❑ Each subsystem (knowledge source) contributes partial solutions.
- ❑ Requires a control component to manage which subsystem acts when.

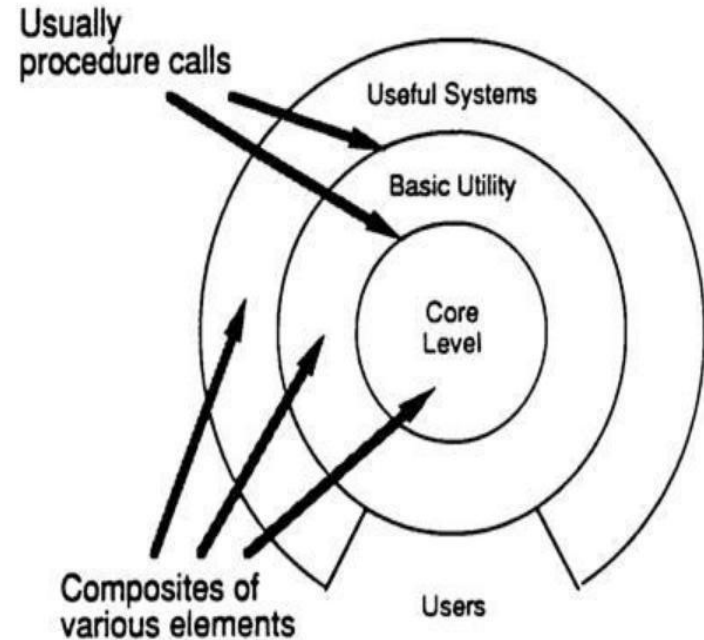


Example: Stock exchange

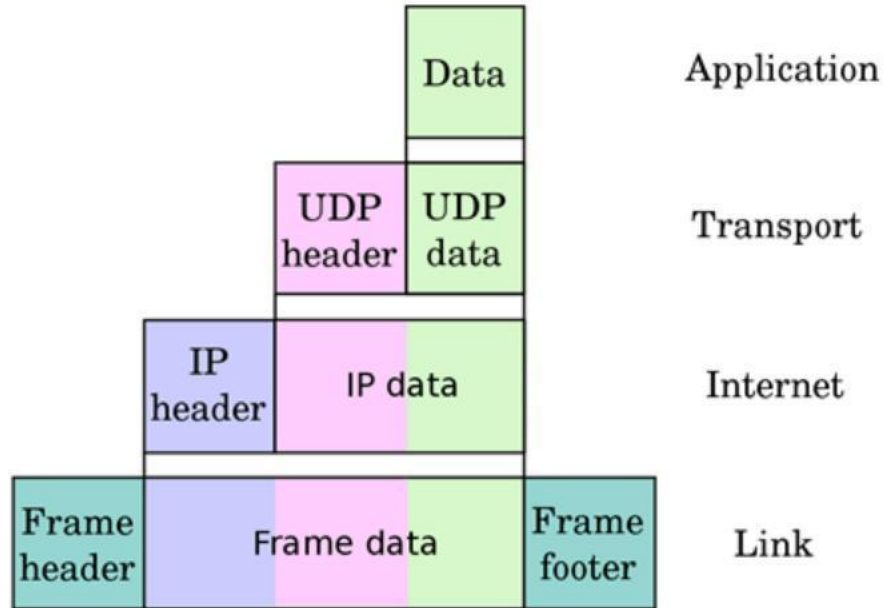


Layered Systems Architecture

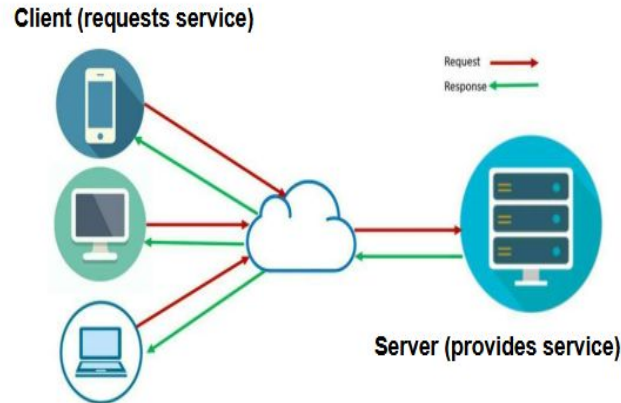
- System is organized into hierarchical layers
- Each layer has a specific role and interacts only with adjacent layers.
- Easy to swap or upgrade individual layers without affecting others.



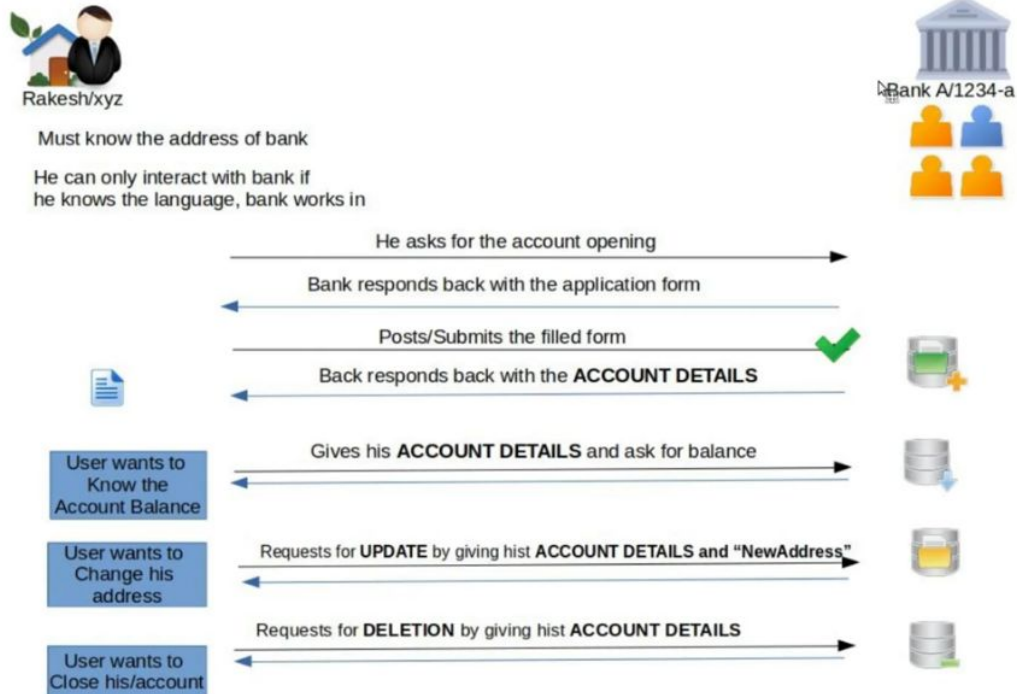
Example: Internet Protocol Suite



- **Divides system into clients and servers:**
 - Clients send requests; servers process and respond
- **Centralized control and resource management:**
 - Servers handle business logic, security, and data storage.
- Clients can be software that depends on a shared database/service.



Example: online banking system



What architecture pattern would you choose and why?

- ☐ Weather app like the Weather Channel
- ☐ Email service like Outlook or Gmail
- ☐ Online banking service like Bank of America
- ☐ Online multi-faceted store like Amazon

What architecture pattern would you choose and why?

System	Best Matching Pattern	Primary Justification
1. Weather App	Pipe & Filter	Unidirectional, sequential data processing from raw sensory ingest to transformed output.
2. Email Service	Client-Server	Persistent, bidirectional protocol-based communication between clients and centralized stateful mail stores.
3. Online Banking	Layered	Strict, unidirectional separation of concerns with enforced security, validation, and audit boundaries.
4. Multi-Faceted Store	Microservices	Independent scalability, deployment, and operational boundaries across distinct business subdomains.

Architecture Considerations

As an architect, consider

□ Level of Abstraction

- Components and their interconnections (communication/apis)

□ Separation of concerns

- High cohesion – light relationship within a component
- Low cohesion – interconnection between the components

□ Modularity

- Decomposable designs, composable components
- Localized changes in case of any requirement changes
- Span of impact (how far can an error spread)

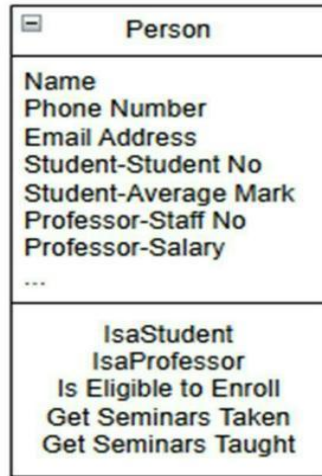
High Cohesion

Cohesion:

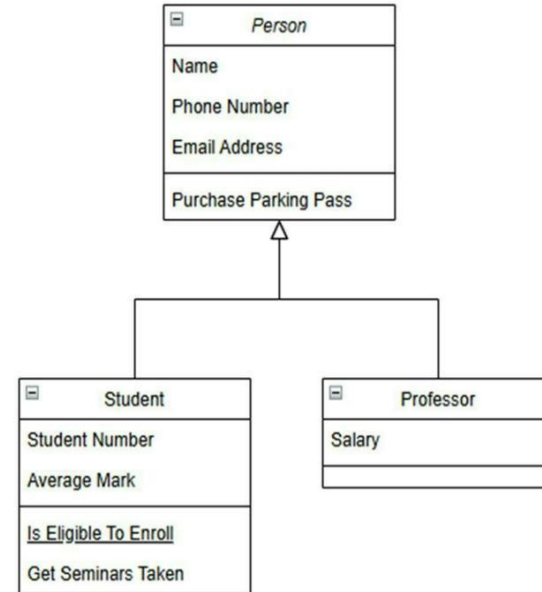
How closely the operations in a module are related and belong together

- High cohesion means a component of a system has a clear purpose and scope
- Strong relationships within a component improve clarity and understanding
- A component with good abstraction usually has strong internal cohesion

Example – low vs high Cohesion



Low Cohesion - All jammed into one class



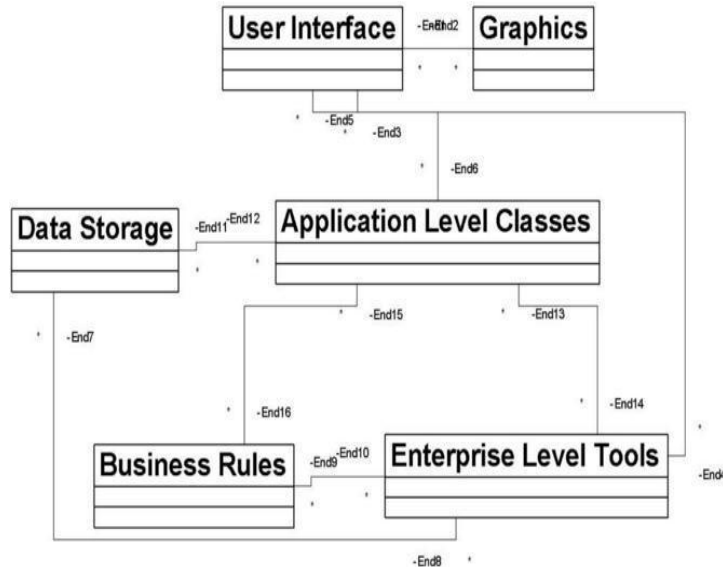
High Cohesion - operations are delegated to dedicated services.

Loose Coupling

Coupling: Level of interconnection between any two components

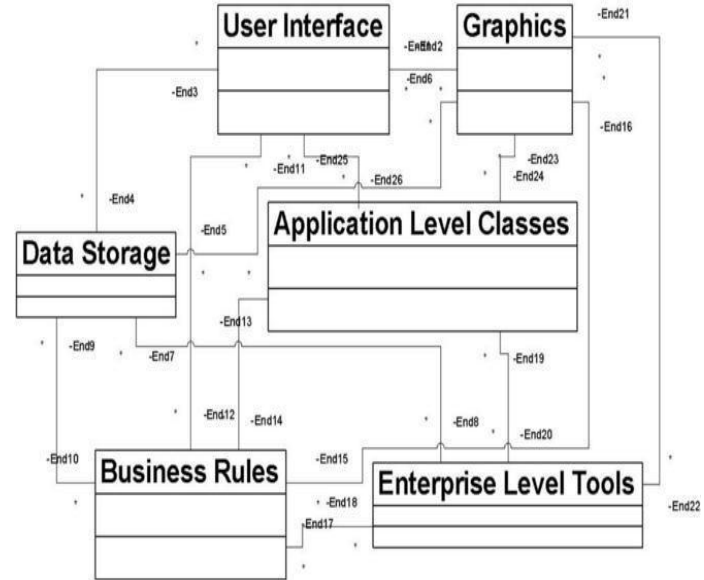
- Components that are **loosely coupled** (or uncoupled) are
 - are easier to work with independently.
 - easier to debug in the system
- Modules that are **tightly coupled** are,
 - harder to work with them separately
 - (consider development, testing, ...)

Example: Lose vs tight Coupling



Loosely Coupled -

- Fewer, structured dependencies
- Limited direct knowledge



Tightly Coupled -

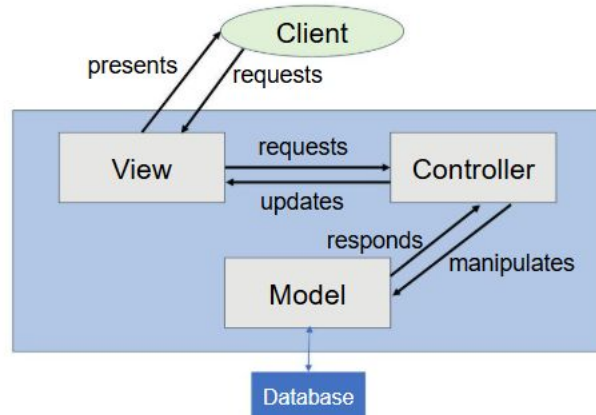
- Point-to-point (spaghetti dependencies)
- High ripple effect

Architectural considerations

- ❑ **System understanding:** interactions between modules
- ❑ **Reuse:** high level view shows opportunity for reuse
- ❑ **Construction:** breaks development down into work items; provides a path from requirements to code
- ❑ **Evolution:** high level view shows evolution path
- ❑ **Management :** helps understand items and track progress.
- ❑ **Communication :** provides vocabulary, a picture says a thousand words.

Model View Controller

- Divides a system into three components:
 - **Model:**
 - App data and core functionality
 - **View**
 - Presents data to user /provides user interface
 - **Controller**
 - Handles control flow / mediates between the view and model

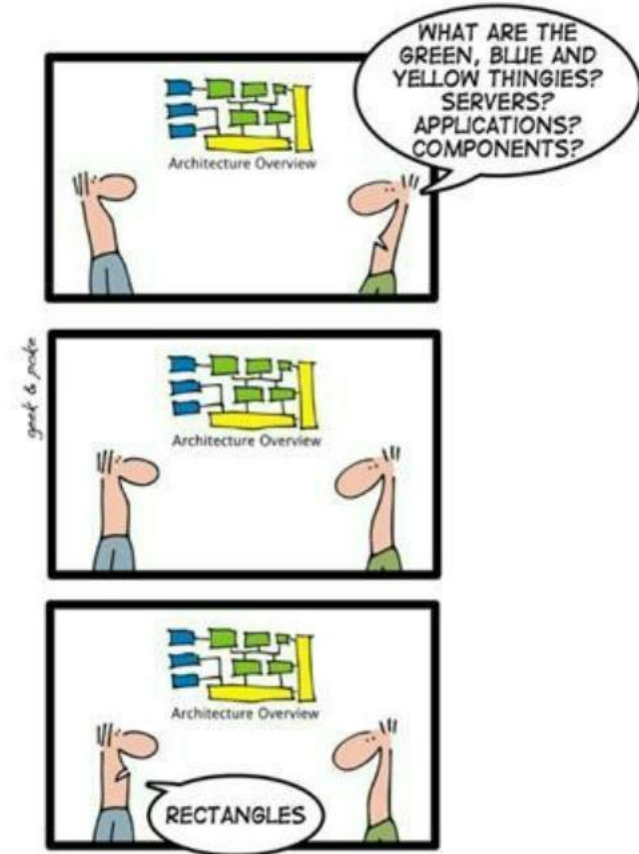


Why Document Architecture?

- Blueprint for the system
 - Artifact for early analysis
 - Primary carrier of quality attributes
 - Key to post-deployment maintenance and enhancement
- Documentation speaks for the architect
 - As long as the system is built, maintained, and evolved according to its documented architecture
- Support traceability.

Guidelines for selecting a notation

- Suitable for purpose
- Often visual for compact representation
- Usually, boxes and arrows
- UML possible (semi-formal), but possibly constraining
 - Note the different abstraction level – Subsystems or processes, not classes or objects
 - Formal notations available
- Decompose diagrams hierarchically and in views
- Always include a legend
- Define precisely what the boxes mean
- Define precisely what the lines mean
- Do not try to do too much in one diagram
- Each view of architecture should fit on a page
- Use hierarchy



An example of Architecture document

1. Introduction

1. Purpose
2. Scope
3. Definitions, Acronyms and Abbreviations
4. References

2. Architectural Representation

3. Architectural Goals and Constraints

4. Use-Case View

1. Architecturally-Significant Use Cases

5. Logical View

1. Architecture Overview – Package and Subsystem Layering
2. Process View
 1. Processes
 2. Process to Design Elements
 3. Process Model to Design
 4. Model Dependencies
 5. Processes to the Implementation

6. *Deployment View*

1. External Desktop PC
2. Desktop PC
3. Registration Server
4. Course Catalog
5. Billing System

7. *Performance*

8. *Quality*

A good sample SA document is [here](#).



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2 - Lecture 2-3

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Security Design and Architecture

Secure Architecture Design – Overview

- **Purpose:** Embed security principles into architectural decisions to mitigate design-level vulnerabilities.
- **Scope:** Applies to software components, data flows, deployment environments, and integration points.
- **Key Goals:**
 - Reduce attack surface
 - Ensure separation of concerns
 - Support defense-in-depth strategies
 - Align with threat models and security requirements

Key Secure Architecture Techniques

❑ Security by Design Patterns:

- Input validation layers
- Secure communication channels (TLS, mTLS)
- Encapsulation and service boundaries
- Divide system into subsystems and create trust boundary
- Components falling in same trust boundaries are marked mutually trusted
- Consider assets at Asset origin point, during transit, at rest and in use

❑ Architecture Review Checkpoints:

- Are all sensitive components isolated?
- Are APIs authenticated, rate-limited, and validated?
- Is encryption used where needed (data in transit/rest)?
- Are default configurations secure?

Security Design and Architecture

Security Objective from example $\Rightarrow$ Patient data should be confidentiality, integrity & replay protected.

Asset derived from example $\Rightarrow$ Patient Data

Protection required from $\Rightarrow$ Other Patients, Insider threat (Malicious hospital employee)

Patient data origin point $\Rightarrow$ Hospital App/Website/Front desk

Patient data during transit $\Rightarrow$ Over network to Hospital database

Patient data at rest $\Rightarrow$ in storage for example in hard disk

Patient data at use $\Rightarrow$ Hospital staff and patient wish to see patient data

Security Design and Architecture

Trust Boundary Definition $\Rightarrow$ Patient and legitimate hospital employee for example doctors and nurses are within trust boundary. Other patient and rest of the hospital employees are out of trust boundary.

Design Protection $\Rightarrow$ provision user credentials for patients & doctors.

Implement patient data access policy for example Doctor may have read and edit permission, others including patient read permission.

Outputs from Architecture Phase

- ❑ **Updated Architecture Diagrams** with security annotations (e.g., trust boundaries)
- ❑ **Security Architecture Review Report** documenting decisions, risks, and mitigations
- ❑ **Change Requests** for identified gaps
- ❑ **Architecture Threat Map** mapping each high-value component to its threat profile

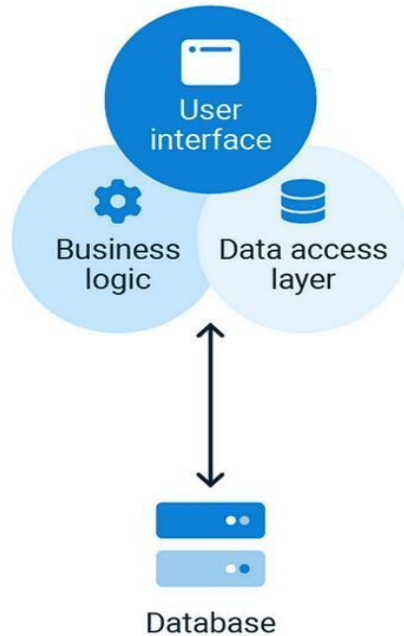
MICROSERVICES

MICROSERVICES

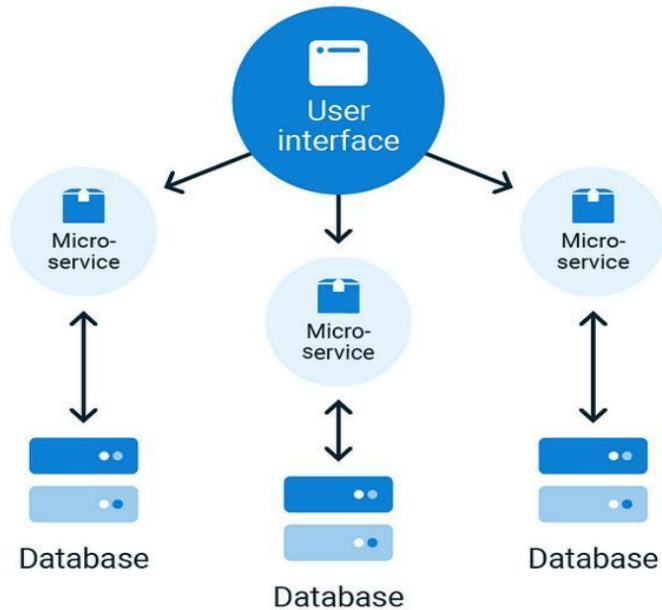
- Micro services are a set of services that act together to make a whole application operate.
- This architecture utilizes APIs to pass information, such as user queries or a data stream, from one service to another.
- Examples: Amazon, Netflix, Uber, ...

MICROSERVICES

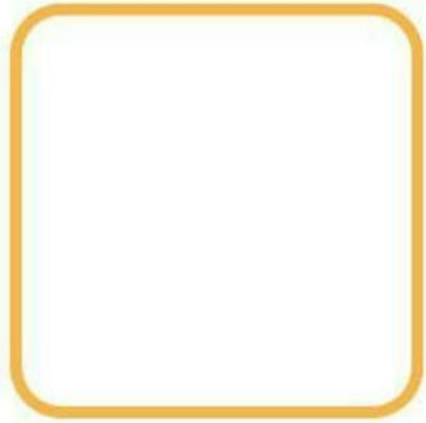
Monolithic Architecture



Microservice Architecture

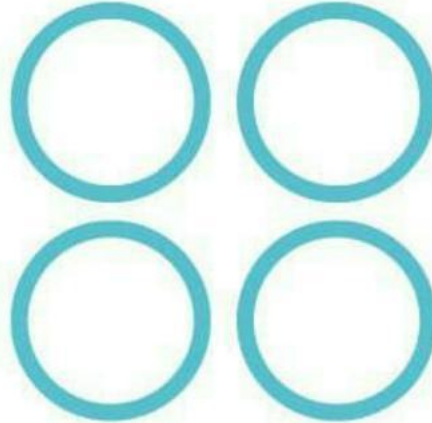


Monolithic vs. Service-based vs. Microservice



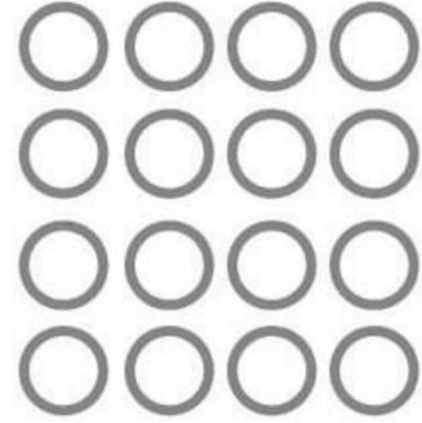
MONOLITHIC

Single Unit



SOA

Coarse- grained



MICROSERVICES

Fine-grained

Reasons to break up the monolith

- ❑ Faster deployment – ship features faster and safer
- ❑ Update certain components more frequently than others.
- ❑ Certain components need different scale requirements or should be developed in a different technology
- ❑ The codebase has gotten too big and complex.
- ❑ Scalability
- ❑ Target security concerns
- ❑ Embrace uncertainty, automation, and faults
- ❑ Better alignment with organization structure

Microservices



UBER

GROUPON®

COMCAST

Netflix Microservices



- User subscriptions
- Banner Ad
- Popular Shows
- Trending Now
- Continue Watching
- My List (saved shows)
- Notifications
- User management
- ...

Why Can't Netflix Use a Monolithic Architecture?



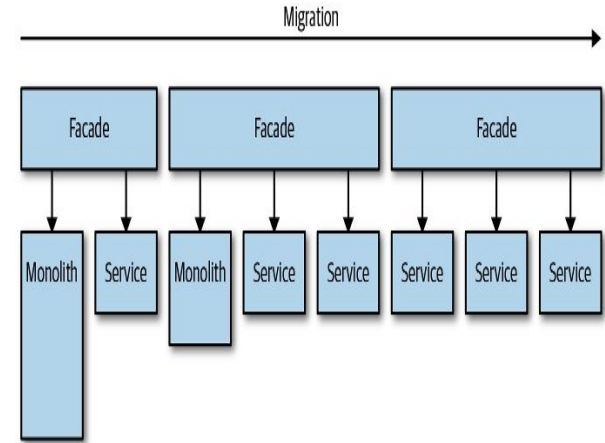
- Require architecture that can handle **various computational demands**
- Need scalability: must support **millions of users** worldwide
- Need fault tolerance to maintain a **seamless user experience**
- New features and improvements need to be **rolled out rapidly**

Methodology to break up a monolith

Two major patterns to move from a monolithic application to microservices:

Strangler pattern

- **New services or existing components are implemented as microservices**
- A **facade** or **gateway** routes user requests to the correct application
- Over time, more and more features /services/functions are moved to the new architecture until the monolithic application has been entirely transformed into a microservices application (and the legacy system is eventually "strangled" and is no longer necessary - can safely be retired.)
- During this process, update the gateway to redirect requests to the new functionality.

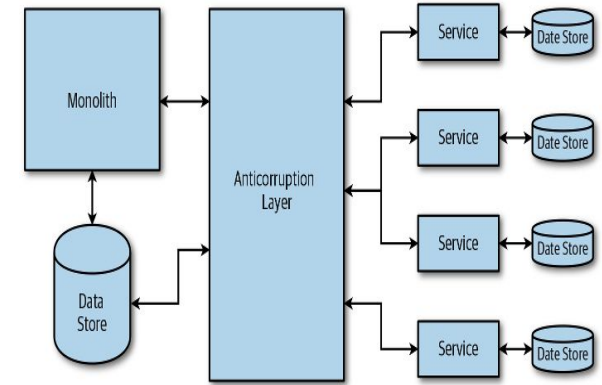


<https://docs.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>

Methodology to break up a monolith

Anti-Corruption Layer pattern

- This is similar to the Strangler pattern but is used **when new services need to access the legacy application**. The layer then translates the concepts from existing app to new, and vice versa
- This pattern is used to implement a **facade or adapter** between components that don't share the same semantics
- The purpose of the anticorruption layer is to translate the request from one component to another; for ex, **implementing protocol or schema translations**
- To implement this, you design and create a new API in the monolith that makes calls through the anticorruption layer in the new service



<https://www.cloudbees.com/blog/anti-corruption-layer-how-keep-legacy-support-breaking-new-systems>

An anti-corruption layer is about mapping one domain onto another so that services that use second domain do not have to be "corrupted" by concepts from the first.

Are Microservices always the right choice ?

Challenges with a Microservices Architecture

❑ Increased Operational Complexity - Managing and deploying multiple services

Example:

In a fintech platform, managing independent microservices for payments, authentication, user management, and analytics requires maintaining separate deployment pipelines and health checks. Any misconfiguration in one service (e.g., wrong environment variable) could cause cascading failures during deployment

❑ Inter-Service Communication – Ensuring reliable communication between services

Example:

A microservice for order processing calls an inventory service to check stock. If the inventory service is slow or down, the order service times out. To mitigate this, developers implement **retries with exponential backoff**, but improper configuration can cause a "retry storm," worsening the load on the already degraded service.

❑ Data Consistency and Integrity - Managing transactions across distributed services

Example:

In an e-commerce platform, placing an order involves updating the order service, inventory, and payment services. Using a **Saga pattern**, each service commits locally and compensating transactions are used to roll back if something fails. If the payment service fails after inventory is deducted, the inventory must be rolled back separately.

Challenges with a Microservices Architecture

- ❑ **Testing and Debugging** - End-to-end testing is hard when services depend on each other

Example:

Testing a customer onboarding workflow requires running services like KYC validation, email notification, user provisioning, and analytics. Mocking each service's responses is non-trivial, and bugs often emerge only in staging or production due to real service behavior not matching mocks

- ❑ **Governance and Standardization** – Lack of uniform standards across services

Example:

One team uses REST with camelCase JSON keys (`userId`), while another uses snake_case (`user_id`). When integrating services, the inconsistencies cause serialization/deserialization issues, leading to production bugs and friction during integration.

- ❑ **DevOps and Infrastructure Overhead** - Managing infrastructure for each service

Example:

Each microservice (e.g., notifications, audit logging, billing) requires its own Dockerfile, Helm charts, CI/CD pipeline, monitoring dashboards, and alert rules. The operations team spends more time on infrastructure maintenance than on improving product capabilities

Challenges with a Microservices Architecture

❑ **Team and Organizational Challenges** - Coordination and skill gaps between teams

Example:

A team responsible for the billing microservice decides to upgrade to a new database version. However, they fail to notify the data ingestion team, whose service depends on specific billing data formats. This causes downstream failures during deployment

❑ **Versioning and Backward Compatibility** – Managing breaking changes across services

Example:

A user profile service renames a field from `fullName` to `name`. Several consuming services, including notifications and analytics, break because they were not updated in time. Even with versioned APIs, maintaining compatibility across consumers is difficult.

❑ **Security and Compliance** - Securing communication and enforcing compliance across services

Example:

In a healthcare system, services like patient data, prescription management, and reporting must all comply with HIPAA. If the report-generation service accidentally logs sensitive patient data without encryption or access control, it can lead to a major compliance violation

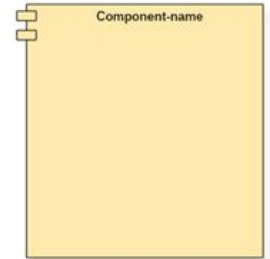
Component Diagrams

Component Definition and Notation

❑ **What is a Component?** A component is a replaceable and executable piece of a system whose implementation details are hidden. Key characteristics:

- Modular: Encapsulates specific functionality
- Replaceable: Can be swapped with other components that provide the same interface
- Black Box: External behavior is defined by interfaces, internal implementation is hidden
- Executable: Participates in system execution

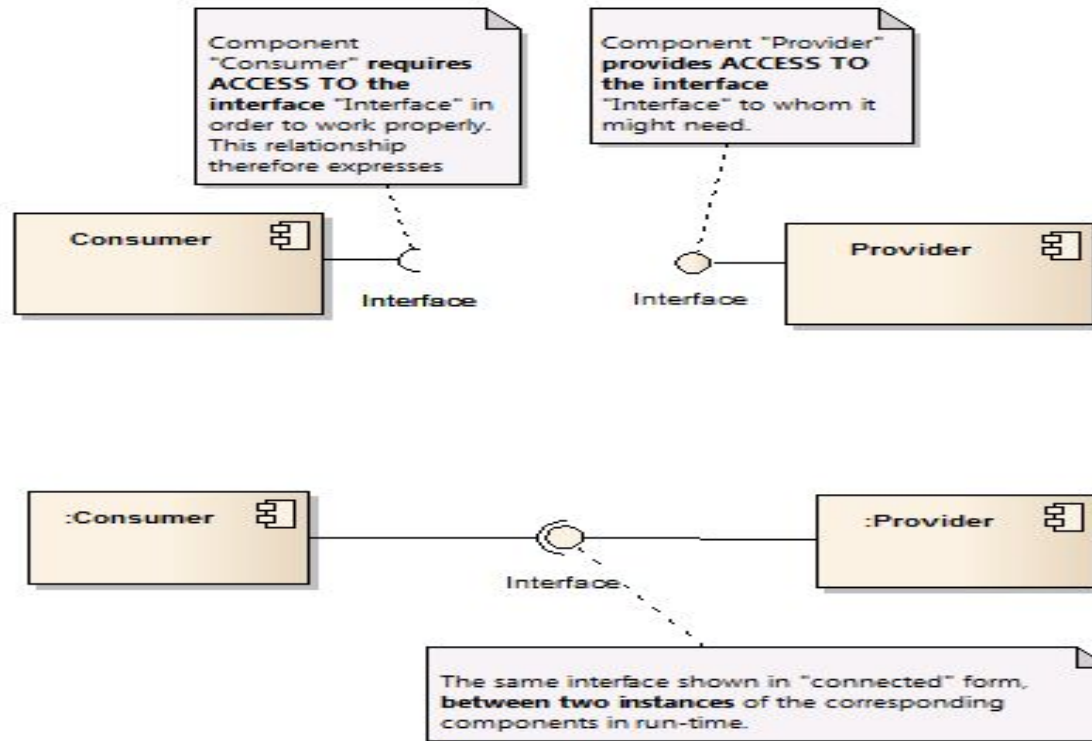
❑ **Component Notation:** In UML, a component is represented by a classifier rectangle with the stereotype <<component>>. The component's name is placed at the center, and an optional icon can be displayed at the upper right corner.



Component Interfaces and Interaction Point

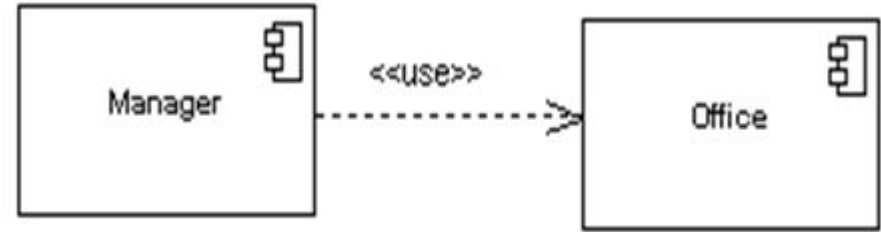
- ❑ **Provided Interfaces:** These indicate the functionality a component offers to other components, shown with a "lollipop" notation.
- ❑ **Required Interfaces:** These indicate the functionality a component needs from other components to operate correctly, shown with a "socket" notation.
- ❑ **Connecting Interfaces:** Provided and required interfaces can be connected using an assembly connector.
- ❑ **Ports:** A port is an interaction point between a classifier and an external environment. It groups a semantically cohesive set of provided and required interfaces.
- ❑ **Port Notation:** A port is denoted as a small square on the boundary of a classifier. When drawn over the boundary, it indicates public visibility, meaning all interfaces used are public. If drawn inside, it is either protected or private

Component Interfaces and Interaction Point



Component Diagram : Usage Dependency

- ❑ **Usage Dependency:** This relationship signifies that one element requires another element for its full implementation. It is a dependency where the client requires the presence of the supplier.
- ❑ **Notation:** A usage dependency is shown as a dashed arrow with a `<<use>>` keyword. The arrowhead points from the dependent component (client) to the one on which it is dependent (supplier).



Component Diagram: Subsystems for Large Systems

- ❑ **Subsystem Definition:** A subsystem is a component base that acts as a decomposition unit for larger systems. It is a logical construct used to break down an extensive system into smaller systems.
- ❑ **Purpose:** This process makes it easy to manage each subsystem efficiently.
- ❑ **Runtime and Connection:** A subsystem cannot be instantiated during runtime, but its contents can be initialized. When subsystems are connected, they create a single system.

Why use and How to Draw Component Diagram

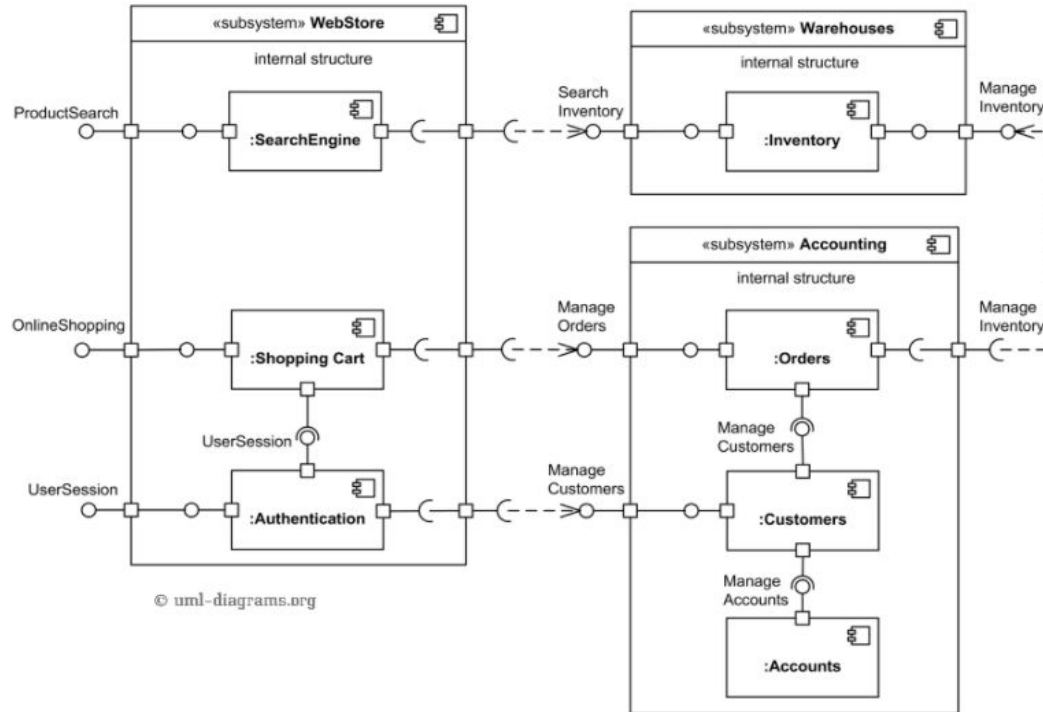
□ Why use Component Diagrams?

- They are used to represent the structure and organization of components during any instance of time.
- They are used for modeling subsystems, which collectively represent the entire working view of any system.
- They help during the testing of a system.
- They visualize the connection between various components.

□ How to Draw a Component Diagram (Key Steps):

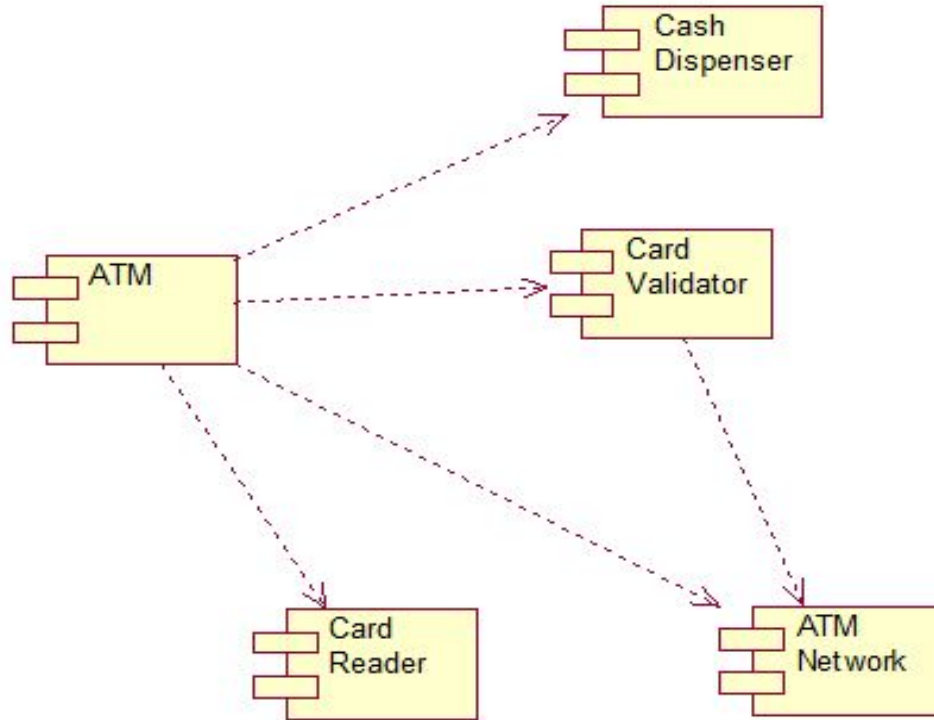
- Before modeling, identify all components within the system and their working.
- Explore each component in depth to understand its connection to other physical artifacts in the system.
- Identify the relationships among various artifacts, libraries, and files.

Real World Examples: Online Shopping System



Online shopping UML component diagram example with three related subsystems - WebStore, Warehouses, and Accounting.

Real World Examples: ATM System





Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2- Lecture 5 - 6

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Software Design

Level Of Abstraction

□ Requirements:

- high-level “what” needs to be done

□ Architecture (High-level design) :

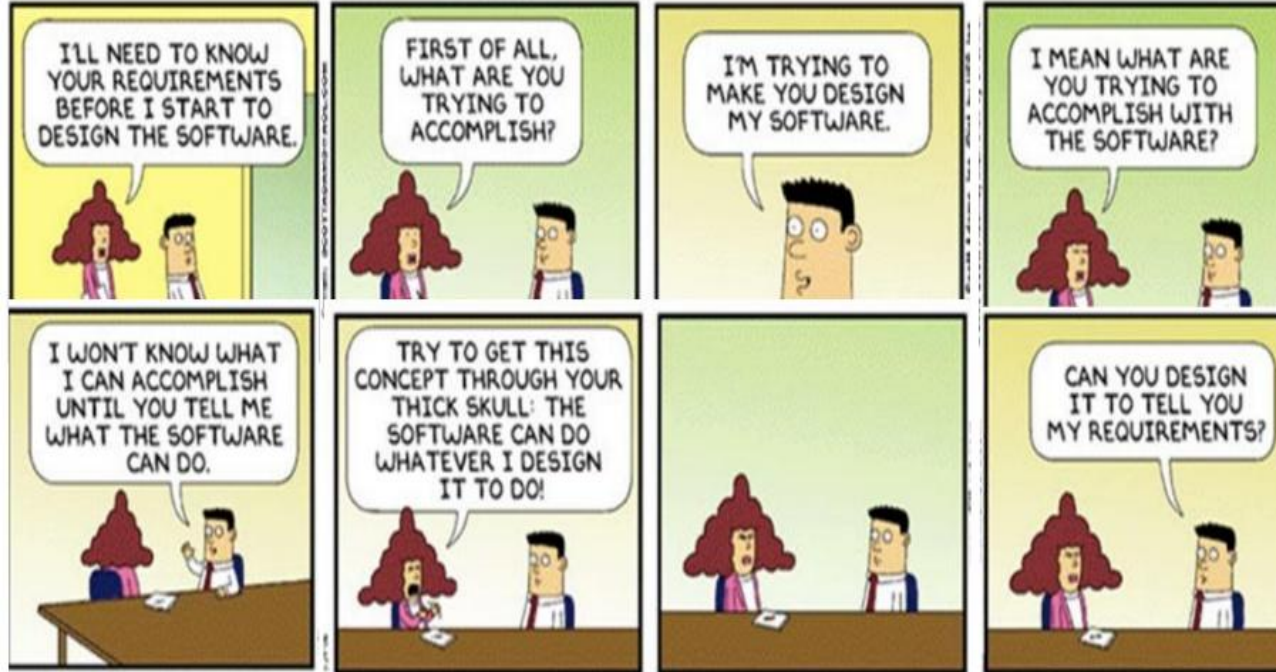
- high-level “how”, mid-level “what”

□ Design:

- Low-level design, e.g. design patterns

□ Code :

- low-level “how”



Software Design Principles

- Further decomposition (post architecture) of the components being developed if necessary.
- Description of the behavior of components/sub-systems identified as part of Architectural design
- Description of how the interfaces will actually be realized using the appropriate algorithms and data structures
- Description on how the system will facilitate interaction with the user through the user interface
- The use of appropriate structural and behavioral design patterns & consideration of maintenance and reuse as couple of its goals

Techniques that enable design

- Abstraction : Focus on essential properties
- Modularity, Cohesion and Coupling
 - Modularity is the degree or the extent to which the larger module can be decomposed
 - Cohesion is the glue that keeps the components together. Cohesion which is the extent to which the component/modules are dependent on/fit into or are related to each other, when trying to address a specific responsibility (Strong cohesion is good)

Techniques that enable design

- There are seven levels of cohesion of increasing strength:
- **Coincidental cohesion** : With coincidental cohesion, elements are grouped into modules in a haphazard way. There is no significant relation between the elements.
- **Logical cohesion** : With logical cohesion, the elements realize tasks that are logically related. One example is a module that contains all input routines. These routines do not call one another and they do not pass information to each other. Their function is just very similar.
- **Temporal cohesion** : A typical example of this type of cohesion is an initialization module. The various elements of it are independent but they are activated at about the same point in time.

Techniques that enable design

- **Procedural cohesion** :A module exhibits procedural cohesion if it consists of a number of elements that have to be executed in some given order. For instance, a module may have to first read some datum, then search a table, and finally print a result.
- **Communicational cohesion** :This type of cohesion occurs if the elements of a module operate on the same (external) data. For instance, a module may read some data from a disk, perform certain computations on those data, and print the result.
- **Sequential cohesion** :Sequential cohesion occurs if the module consists of a sequence of elements where the output of one element serves as input to the next element.
- **Functional cohesion**: In a module exhibiting functional cohesion all elements contribute to one single function of that module. Such a module often transforms a single input datum into a single output datum. The well-known mathematical subroutines are a typical example of this. Less trivial examples are modules like 'execute the next edit command' and 'translate the program given'.

Coupling

- Coupling indicates how strongly the modules are connected to other modules. (Loose coupling is good)
- The following types of coupling can be identified (from tightest to loosest):
 - **Content coupling** : With content coupling, one module directly affects the working of another module. Content coupling occurs when a module changes another module's data or when control is passed from one module to the middle of another (as in a jump). This type of coupling can, and should, always be avoided.
 - **Common coupling** : With common coupling, two modules have shared data. The name originates from the use of COMMON blocks in FORTRAN. Its equivalent in block-structured languages is the use of global variables.
 - **External coupling** : With external coupling, modules communicate through an external medium, such as a file.

Food for thought

□ What is the type of cohesion for Input routines?

Ans: Logical

□ What is the type of cohesion for Initialization sequence?

Ans: Temporal

□ What is the type of cohesion for Read and print?

Ans: Procedural

Coupling

- ❑ **Control coupling** :With control coupling, one module directs the execution of another module by passing the necessary control information. This is usually accomplished by means of flags that are set by one module and reacted upon by the dependent module.
- ❑ **Stamp coupling** :Stamp coupling occurs when complete data structures are passed from one module to another. With stamp coupling, the precise format of the data structures is a common property of those modules.
- ❑ **Data coupling** :With data coupling, only simple data is passed between modules.

Techniques that enable design

- **Information hiding:** Design involves a series of decisions and for each such decision, we need to consider who needs to know & who can be kept in the dark. This is done through:
- **Encapsulation** :An information hiding strategy which hides data and allows access to the data only through specific functions
 - **Separation of interface and implementation:** An information hiding strategy which involves defining a component by specifying a public interface (known to the clients) but separating the details of how the component is actually realized

Techniques that enable design

- **Limiting Complexity:** Complexity refers to the amount of effort for building a solution.
 - Can be Intra – modular (within a module) or Inter – modular (between modules)
 - Higher value => Higher complexity => Higher effort required (= worse design)

- **Hierarchical structure :** Views the whole structure as a hierarchy.

Key issues

- ❑ Concurrency
- ❑ Non-functional requirements
- ❑ Data persistence
- ❑ Event handling (Callback mechanism)
- ❑ Error, Exception handling, Fault tolerance
- ❑ Distribution of components
- ❑ Interaction and Presentation

Design Methods

- Detailed design methods support us in decomposing the components representing the system requirement into components/subcomponents well.
 - Booch methodology - is a method for object-oriented software development.
 - Fusion - is a systematic software development method for object-oriented software development (Developed at Hewlett-Packard Laboratories in Bristol, England)

- **Data Flow Diagrams (DFD):** Developed by Yourdon and Constantine in early 70s
 - It is a two-step process:
 - Structured Analysis (SA), resulting in a logical design, drawn as a set of data flow diagrams
 - Structured Design (SD) transforming the logical design into a program structure drawn as a set of structure charts
 - Data Flow Diagrams (DFD) are graphical representations of a system that illustrate the flow of data within the system. DFDs can be divided into different levels, which provide varying degrees of detail about the system.

What is UML ?

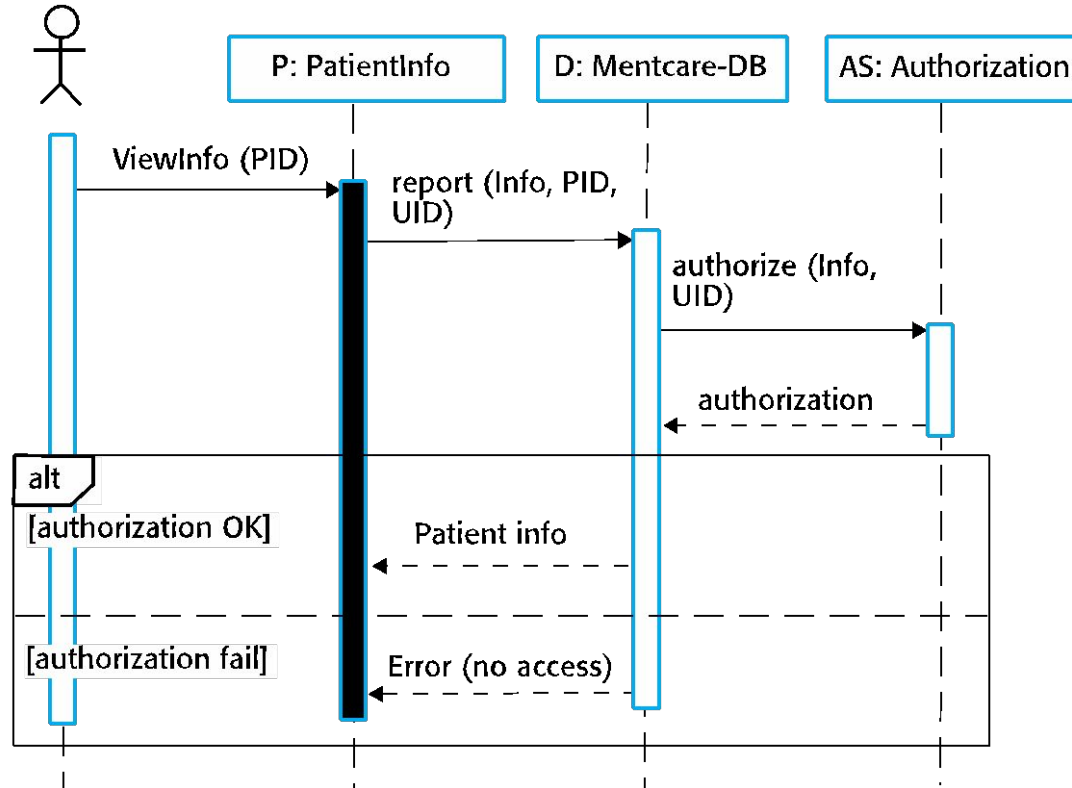
- UML refers to Unified Modelling Language.
- Developed in the mid 90's improved since then.
- Standardization notation for modelling object oriented systems.
- A collection of models for different viewpoints :
 - Use case diagram
 - Component diagram
 - Class and object diagram
 - Sequence diagram
 - State chart diagram
 - ...

Sequence Diagrams

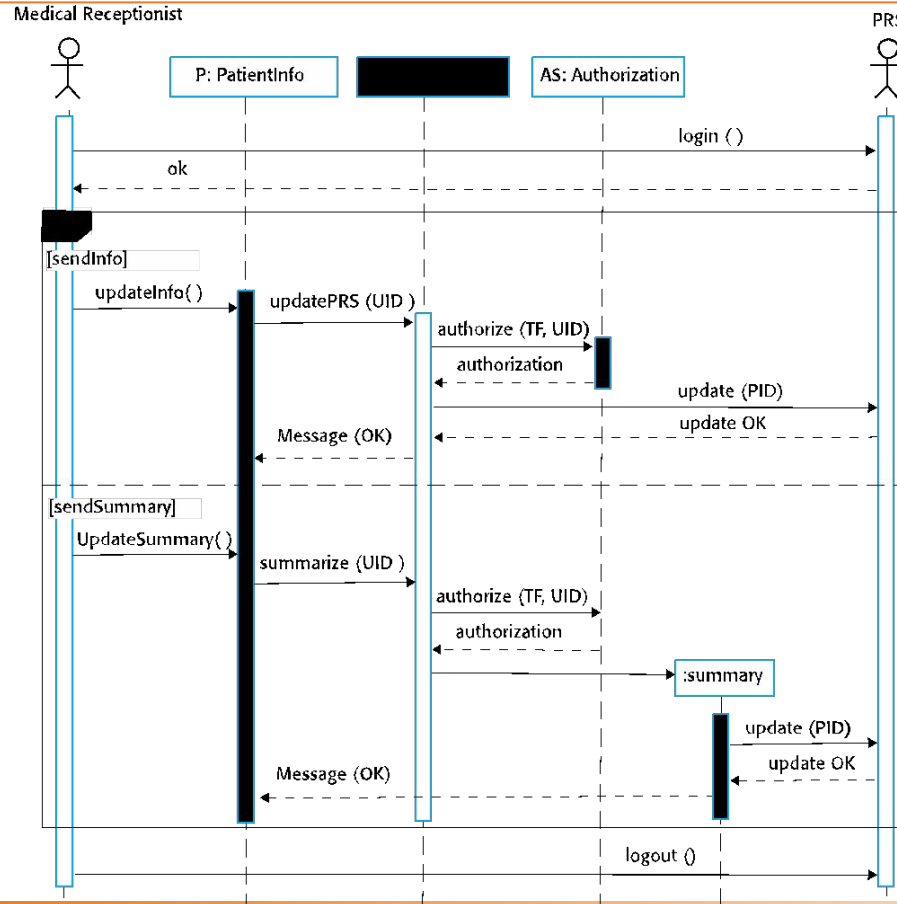
- Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- Interactions between objects are indicated by annotated arrows.

Sequence diagram for view patient information

Medical Receptionist



Sequence diagram for transfer data





Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2- Lecture 5 - 6

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Modularity

Why Do API and ABI Matter?

- ❑ **System Integration:** APIs and ABIs define how different components of a software system communicate and interact. APIs govern source-level interaction, while ABIs handle binary-level interoperability between compiled modules or external libraries.
- ❑ **Modular Architecture:** They act as contracts between modules—enabling independent development, easier debugging, testability, and parallel development workflows. For example, a UI module can call an API of a business logic module without needing to understand its internals.
- ❑ **Cross-Platform and Cross-Language Interfacing:** ABI compliance ensures that compiled code (e.g., shared libraries) works across different tools and platforms. This is vital in environments like Linux, where applications dynamically link to system libraries.
- ❑ **Example:** A database client written in Python may use a C-based driver library (via ABI), while exposing a higher-level API to application logic. This layered separation maintains abstraction and compatibility across updates.

What Is an API?

- **Definition:** An API (Application Programming Interface) is a formal contract that specifies how different software components interact. It defines available functions, expected inputs, returned outputs, error codes, and communication protocols. By abstracting implementation details, APIs allow modules to evolve independently.
- **Role:** APIs act as the boundary between modules, hiding complexity and presenting a simple interface. This separation allows teams to work in parallel: one team implements functionality behind the API, while another consumes it without knowing internal logic.

Why API Definition Matters

- ❑ **Modularity & Encapsulation:** Well-defined APIs enforce encapsulation by limiting access to only necessary behavior. This minimizes unintended dependencies and makes the codebase more robust against changes.
- ❑ **Reusability & Collaboration:** Clear API contracts enable reuse of libraries and components across projects. When teams share an API spec, they can develop client and server code independently, accelerating delivery timelines.
- ❑ **Maintainability:** Confined behind stable interfaces, internal implementations can change without impacting consumers. Good versioning practices further ensure that breaking changes are managed gracefully.

Key Aspects of API Design

- ❑ **Clarity:** Use descriptive names and consistent parameter ordering. For example, choose `createUserProfile()` over ambiguous names like `userOp()`. Clear naming reduces integration errors and enhances readability.
- ❑ **Simplicity:** Expose only essential functions; avoid bloating the interface. A minimal surface area means fewer integration points to test and maintain.
- ❑ **Error Handling:** Standardize error codes and messages so clients can programmatically respond to failures. For instance, returning structured JSON with `errorCode` and `message` fields helps clients detect and handle issues uniformly.
- ❑ **Versioning Strategy:** Embed version numbers in API paths or use semantic versioning in headers. For example, `/v1/users` vs `/v2/users` lets clients opt into new behavior without breaking existing integrations.

Achieving Modularity through API Design

- ❑ **Loose Coupling:** Depend only on abstract interfaces rather than concrete implementations. This allows swapping modules without rewriting dependent code.
- ❑ **High Cohesion:** Group related functionality in the same module and expose it via cohesive API calls. For example, a UserService API may offer create, read, update, and delete operations, rather than mixing unrelated operations.
- ❑ **Information Hiding:** Prevent consumers from relying on internal data structures by providing only accessor methods. This reduces the risk of breaking clients when internals change.
- ❑ **Extensibility:** Use plugin or dependency injection patterns where the main application consumes modules implementing a common API, enabling dynamic loading of features.

Types of APIs – Function/Library Level

- ❑ **Static & Dynamic Libraries:** In Linux, dynamic libraries (.so files) expose symbols via headers. Clients link at compile time (static) or load at runtime (dynamic), using prototypes from header files.
- ❑ **Explanation:** The `open()`, `read()`, and `close()` functions form a simple API for file operations. Their prototypes in `fcntl.h` and `unistd.h` define the contract.

// POSIX file I/O example

```
#include <fcntl.h>
#include <unistd.h>
```

```
int fd = open("data.txt", O_RDONLY);
if (fd < 0) {
    perror("open");
    return -1;
}
```

```
char buffer[128];
ssize_t n = read(fd, buffer,
    sizeof(buffer));
if (n < 0) perror("read");
close(fd);
```

Types of APIs – IPC-Level APIs

- ❑ **Mechanisms & APIs:** POSIX IPC includes message queues, shared memory, and semaphores—each with its own API.
- ❑ **Design Considerations:** When defining IPC APIs, specify synchronization semantics (e.g., mutexes), message formats, and access permissions to avoid data races and security issues.

```
// POSIX shared memory example  
#include <sys/mman.h>  
#include <fcntl.h>  
#include <sys/stat.h>  
#include <unistd.h>
```

```
int shm_fd = shm_open("/myshm",  
O_CREAT | O_RDWR, 0666);  
ftruncate(shm_fd, 4096);
```

```
void *ptr = mmap(0, 4096, PROT_READ  
| PROT_WRITE, MAP_SHARED, shm_fd,  
0);
```

```
strcpy((char*)ptr, "Hello, IPC!");
```

Types of APIs – Network & REST APIs

- ❑ **Protocols & Formats:** RESTful APIs use HTTP/HTTPS and data formats like JSON or XML for interoperability. A clear resource-based URL structure improves discoverability.
- ❑ **Design Patterns:** Use consistent HTTP methods (GET, POST, PUT, DELETE). Document error responses (e.g., 404 Not Found, 400 Bad Request) so clients can handle them programmatically.

```
# OpenAPI spec snippet for "get user" endpoint
oas: 3.0.0
paths:
  /v1/users/{id}:
    get:
      summary: Get user by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: User found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
```

Synchronous vs Asynchronous APIs

- **Synchronous API:** The caller waits (blocks) until the operation is completed. Execution continues only after the response is received.
 - **E.g. Linux `read()` and `write()` syscalls (C/C++):**
 - Classic POSIX I/O is blocking unless marked otherwise.
 - If you `read()` a file or socket, your thread waits until data is available.
 - Example use: `int bytes = read(fd, buffer, size);`
 - **Python requests library:**
 - `requests.get()` waits for response.
 - Widely used in synchronous REST clients and scripts.

Synchronous vs Asynchronous APIs

- **Asynchronous API:** The caller invokes the operation and continues execution immediately without waiting. The response is handled later (e.g., via callbacks, polling, or event loops).
 - **E.g. Node.js HTTP Server (JavaScript backend):**
 - Everything is async by default using callbacks or promises.
 - Enables handling 1000s of connections using event loop.
 - **Python asyncio + aiohttp:**
 - Enables writing async web clients and servers.
 - Non-blocking: handles thousands of requests in Python backend (e.g., FastAPI, Quart).

Design Considerations

- Use **synchronous APIs** when:
 - Simplicity is key.
 - Operations are short-lived or internal.
- Use **asynchronous APIs** when:
 - Dealing with high-latency or I/O-bound systems (e.g., network APIs).
 - Scaling large applications (e.g., web servers, microservices).
 - You need responsiveness (e.g., GUI or real-time apps).

API Standards & Specifications

□ **OpenAPI/Swagger:**

Contract-first approach lets teams generate client and server code from a single spec. This reduces manual syncing errors between implementation and docs.

□ **gRPC & Protobuf:** Define services and messages in .proto files, generating strongly-typed code in multiple languages. Protobuf's binary format improves performance for high-throughput systems.

```
// gRPC service example
syntax = "proto3";
package user;
service UserService {
    rpc GetUser (GetUserRequest)
    returns (User);
}
message GetUserRequest {
    string id = 1;
}
message User {
    string id = 1;
    string name = 2;
}
```

Versioning & Compatibility

- ❑ **Backward Compatibility:** Newer API versions should accept requests from older clients without breaking existing functionality. Techniques include adding optional fields and keeping default behaviors unchanged.
- ❑ **Forward Compatibility:** Older API versions should ignore unknown fields sent by newer clients. Protocol buffers handle this gracefully by skipping unknown tags.
- ❑ **Semantic Versioning:** Use MAJOR.MINOR.PATCH: increment MAJOR for incompatible changes, MINOR for backwards-compatible additions, and PATCH for bug fixes.

Introducing ABI (Application Binary Interface)

- **Definition & Importance:** An ABI defines low-level details—calling conventions, data type sizes, symbol names—so that independently compiled binaries can interoperate. Unlike APIs, ABIs cannot change without breaking binary compatibility.
- **Impact:** Changing the layout of a struct or the signature of a function in a library without updating its ABI can cause crashes or undefined behaviour in client applications.

Real-World ABI Breakage Examples

- ❑ **OpenSSL 1.1.0 Update (2016):** Changed internal data structures and removed symbols, breaking compatibility with applications linking to older versions. Many downstream projects had to recompile or patch code to restore compatibility.
- ❑ **Glibc SONAME Bump:** When the GNU C Library increments its SONAME (e.g., libc.so.6 to libc.so.7), binaries linked against the old SONAME no longer find the library at runtime, causing widespread failures until re-linked.
- ❑ **LLVM and Clang Tooling:** Changes in symbol visibility and data layout between minor releases have led some plugins and tools to break unexpectedly, requiring careful ABI management in plugins.

Implications of ABI Breakage

- ❑ **Runtime Failures:** Applications may crash or throw unresolved symbol errors (undefined symbol: xyz) when loading shared libraries with changed ABIs.
- ❑ **Security Risks:** Mismatched ABIs can bypass intended security checks or introduce memory corruption if data layouts differ.
- ❑ **Maintenance Burden:** Frequent ABI breaks force recompilation of dependent packages, complicating release management for distributions (e.g., Linux distros).
- ❑ **Ecosystem Fragmentation:** Divergent ABI versions can lead to “dependency hell” where different components require incompatible library versions.

Language Specifics

- ❑ **APIs Are Universal:** Every language and framework relies on clear interface definitions; only the mechanisms differ (headers vs. module exports vs. IDLs).
- ❑ **ABIs Vary:**
 - **Interpreted Languages** (Python, JS) have no intrinsic ABI at the source level but rely on runtime or FFI (Foreign Function Interface) layers for extension modules.
 - **Compiled Languages** (C/C++, Rust, Go, Java bytecode) define and manage ABIs more explicitly, often with dedicated tooling and versioning schemes.
- ❑ **Best Practices Still Apply:** Regardless of language, you should:
 - Adopt semantic versioning.
 - Provide deprecation paths.
 - Automate compatibility checks (source or binary) in CI pipelines.
 - Document your public interfaces and evolution policies.



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2- Lecture 5 - 6

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Error Handling

Error Handling

- ❑ **Error handling** in software engineering refers to the process of anticipating, detecting, and responding to runtime errors or exceptional conditions in a program to ensure it continues to operate correctly or fails gracefully.
- ❑ It helps maintain software stability, user experience, and system security.

Defining Errors in API Contracts

- ❑ **Error Codes & Enums:** Use enumerated types or constants to define error cases centrally. This ensures consistency across modules and simplifies both implementation and client-side handling.
- ❑ **Structured Responses:** For network or IPC APIs, return structured payloads (e.g., JSON with code, message, details fields) so callers can parse and react programmatically.
- ❑ **Documentation:** Document each error code with its meaning, range of use, and recovery suggestions to guide developers and maintain a clear contract.

Error Propagation & Handling Patterns

- ❑ **Return Code Checks:** In procedural languages, functions return status codes that callers must check. Use helper macros or inline functions to reduce boilerplate and enforce checks.
- ❑ **Exception-Based Flow:** In OO languages, throw exceptions for unrecoverable errors, catching them at well-defined boundaries. This separates normal logic from error-handling code.
- ❑ **Result/Option Types:** Functional languages or modern languages (Rust, Swift) use `Result<T, E>` or `Option<T>` to encapsulate success or error, forcing callers to handle both cases explicitly.
- ❑ **HTTP Status Codes:** REST APIs leverage HTTP status codes (200, 400, 500) alongside JSON error bodies to indicate success or failure semantically.

Logging & Observability

- ❑ **Structured Logging:** Emit logs in JSON or key-value pairs, including fields like timestamp, error_code, module, and stack_trace. This supports centralized parsing and alerting.
- ❑ **Error Levels:** Use severity levels (INFO, WARN, ERROR, FATAL) to distinguish recoverable vs. critical failures. Tailor alerts to critical levels.
- ❑ **Correlation IDs:** Attach unique request or transaction IDs to log entries, enabling end-to-end tracing of errors across distributed or multi-threaded flows.

Error Code Definition Across Modules

- ❑ **Scoped Error Namespaces:** Prefix error codes by module (e.g., AUTH_INVALID_TOKEN, DB_CONN_FAILED) to avoid collisions and clarify origin.
- ❑ **Central Registry:** Maintain a single source of truth (YAML or constants file) for all codes to simplify discovery and prevent duplication.
- ❑ **Versioning Error Codes:** When deprecating or reassigning codes, maintain backward compatibility by marking codes as obsolete but still supported for at least one major release.

Best Practices for Error Handling Design

- ❑ **Fail Fast:** Detect and report errors as early as possible to simplify debugging. Use assertions or invariant checks in development builds.
- ❑ **Graceful Degradation:** Where possible, degrade functionality rather than crashing entirely (e.g., read cached data if live fetch fails).
- ❑ **Consistent Strategy:** Apply uniform error-handling conventions (return vs. exception) across the codebase to minimize cognitive load.
- ❑ **Automated Tests:** Include negative test cases for error paths, ensuring errors are raised and handled as expected.

Example: Error Handling in C

```
typedef enum {  
    ERR_OK = 0,  
    ERR_FILE_NOT_FOUND,  
    ERR_FILE_READ_FAIL,  
    ERR_MEMORY_ALLOCATION  
} ErrorCode;
```

```
ErrorCode read_config_file(const char *filename, char **content) {  
    FILE *fp = fopen(filename, "r");  
    if (!fp) return ERR_FILE_NOT_FOUND;  
  
    fseek(fp, 0, SEEK_END);  
    long size = ftell(fp);  
    rewind(fp);  
  
    *content = malloc(size + 1);  
    if (!*content) {  
        fclose(fp);  
        return ERR_MEMORY_ALLOCATION;  
    }  
  
    if (fread(*content, 1, size, fp) != size) {  
        fclose(fp);  
        free(*content);  
        return ERR_FILE_READ_FAIL;  
    }  
    (*content)[size] = '\0';  
    fclose(fp);  
    return ERR_OK;  
}
```

```
int main() {  
    char *data = NULL;  
    ErrorCode result = read_config_file("config.txt", &data);  
  
    switch (result) {  
        case ERR_OK:  
            printf("Config: %s\n", data);  
            free(data);  
            break;  
        case ERR_FILE_NOT_FOUND:  
            fprintf(stderr, "Error: File not found\n");  
            break;  
        case ERR_MEMORY_ALLOCATION:  
            fprintf(stderr, "Error: Memory allocation failed\n");  
            break;  
        case ERR_FILE_READ_FAIL:  
            fprintf(stderr, "Error: File read failed\n");  
            break;  
    }  
    return result;  
}
```

Example: Error Handling in Python

```
logging.basicConfig(level=logging.INFO)
```

```
class ConfigError(Exception):  
    pass
```

```
def read_config_file(filename):  
    try:  
        with open(filename, 'r') as f:  
            data = json.load(f)  
            if "setting" not in data:  
                raise ConfigError("Missing 'setting' key")  
            return data  
    except FileNotFoundError:  
        logging.error("Configuration file not found: %s", filename)  
        raise  
    except json.JSONDecodeError as e:  
        logging.error("Invalid JSON in config file: %s", e)  
        raise  
    except ConfigError as e:  
        logging.error("Config error: %s", e)  
        raise
```

```
def main():
```

```
    try:
```

```
        config = read_config_file("config.json")  
        logging.info("Configuration loaded: %s",  
config["setting"])
```

```
    except Exception as e:
```

```
        logging.error("Startup failed: %s", e)
```

```
if __name__ == "__main__":  
    main()
```

Useability Engineering

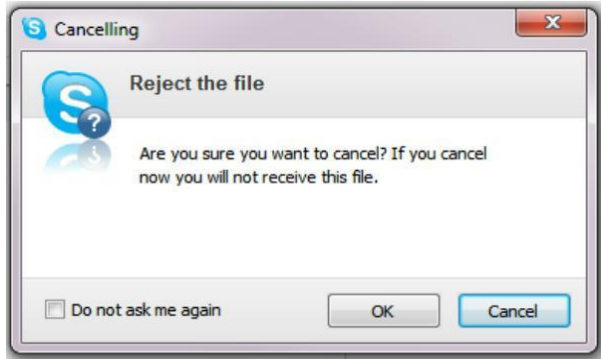
Usability Engineering

- **Usability engineering** in software engineering is the systematic process of designing and evaluating software interfaces to ensure they are easy to learn, efficient to use, and satisfying for the intended users.
- It focuses on improving the user experience (UX) by aligning software functionality with human needs and capabilities.

Which of these designs is the easiest to read ?



Take a chance and guess what they mean !?



Ok, innocent enough. But can anything *bad* happen ?

Well.... Yes.

- ❑ **Human error in interface:** During a shift change drill, an employee **selected the live alert** instead of the test message—likely due to confusing UI layout.
- ❑ **No safeguard separation:** There was insufficient differentiation between “test” and “real” modes—no clear firewall or confirmation step .
- ❑ **No quick cancellation:** Once triggered, the system could not retract the alert—authorities relied on **social media** and manual mechanisms, taking **38 minutes** to clarify the error
- ❑ **Organizational gaps:** Investigations later found weak supervision, poor communication protocols, and a lack of inspection controls .



Public Reaction & Behavioral Impact

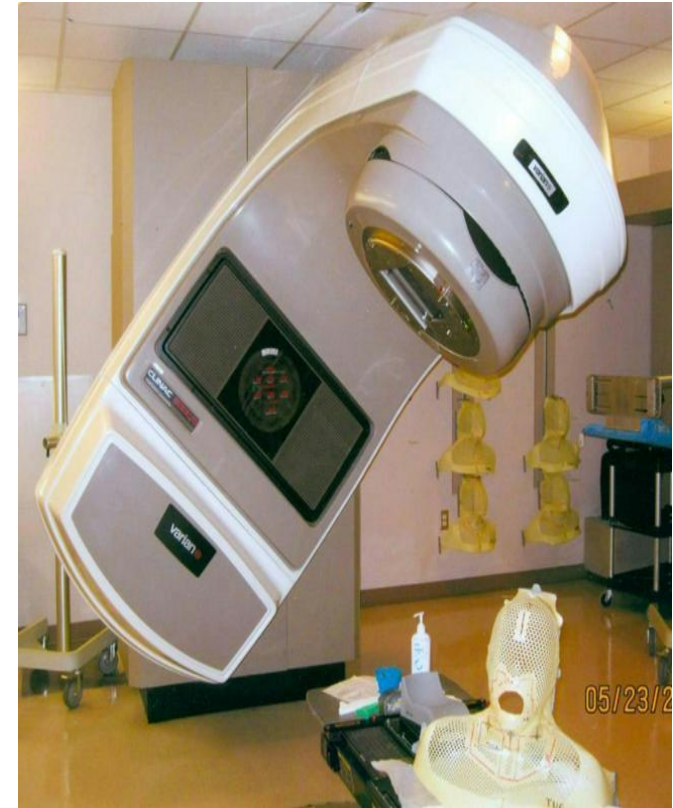
- ❑ **Fear and confusion:** Many believed they were moments from nuclear impact. As one eyewitness said, they thought “they were going to die”
- ❑ **“Social milling” behavior:** Rather than immediate panic, most people **sought verification**—calling or checking social media—before deciding what to do



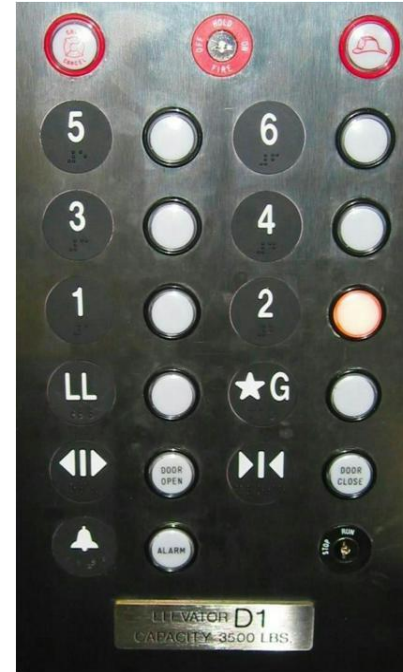
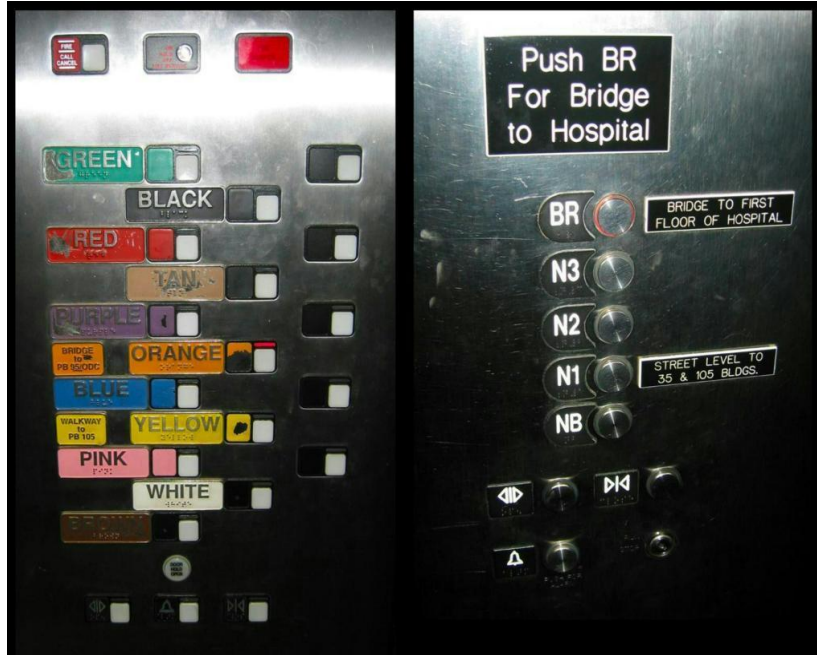
Therac 25

What Went Wrong ?

- ❑ **Race Conditions:** A timing bug allowed operators to rapidly change settings, causing the software to misconfigure the machine—resulting in a high-dose electron beam firing without proper hardware setup
- ❑ **Integer Overflow / Flag Bug:** A control variable tracking hardware readiness overflowed (from 255 back to 0), tricking the system into thinking it was safe to deliver radiation even when critical components weren't in place.
- ❑ **Removal of Hardware Interlocks:** Unlike its predecessors (Therac-6/20), the Therac-25 relied solely on software for safety, eliminating physical safeguards—making software bugs lethal
- ❑ **Operator Overconfidence:** Cryptic error messages like “Malfunction 54” appeared frequently but could be bypassed, leading operators to ignore warnings
- ❑ **Inadequate Testing & Oversight:** Safety checks focused on hardware; software and user interaction were insufficiently tested. Risk assessments underestimated bug risks by several orders of magnitude



Good luck not getting confused !

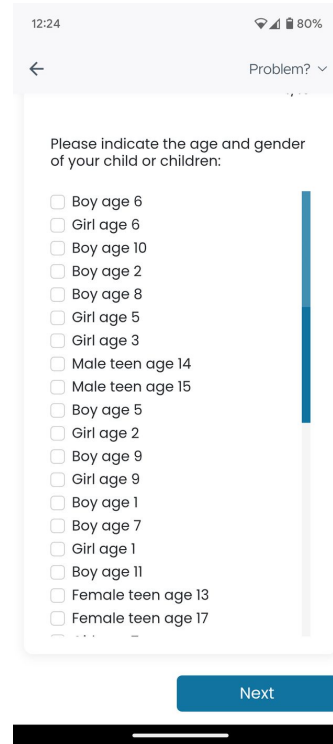




For more examples of bad UI



For more examples visit
[Intentionally bad User Interfaces](#)




How do we test Usability?

- ❑ Intuition?
- ❑ Test on ourselves?
- ❑ Copy interface from other apps ?
- ❑ Not worry about it until someone dies ?

How do you test your features?

- Who do you have test it ?
- What do they do with it?
- What do you measure?
- What is the bar for “good” ?

Test with actual *USERS*

□ Put your app in front of actual people.

- Can they figure it out ?
- How long does it take them?
- Do they need help?

All of these are indicators of how good the usability actually is.

- **How they behave != How you want them to behave**
 - Don't blame the user, blame the design and go fix it.

How do you measure usability ?

□ Imagine your group project

- How do you measure whether it's usable?
- How do you test it with people ?
- How do you know what could be the potential improvements to make?

Low-cost user testing

- Ask a few people to try your app every few weeks.
 - Can they figure it out without help?
 - Give them a task with it... do they do it as you expected?

What problems might be there with this method ?

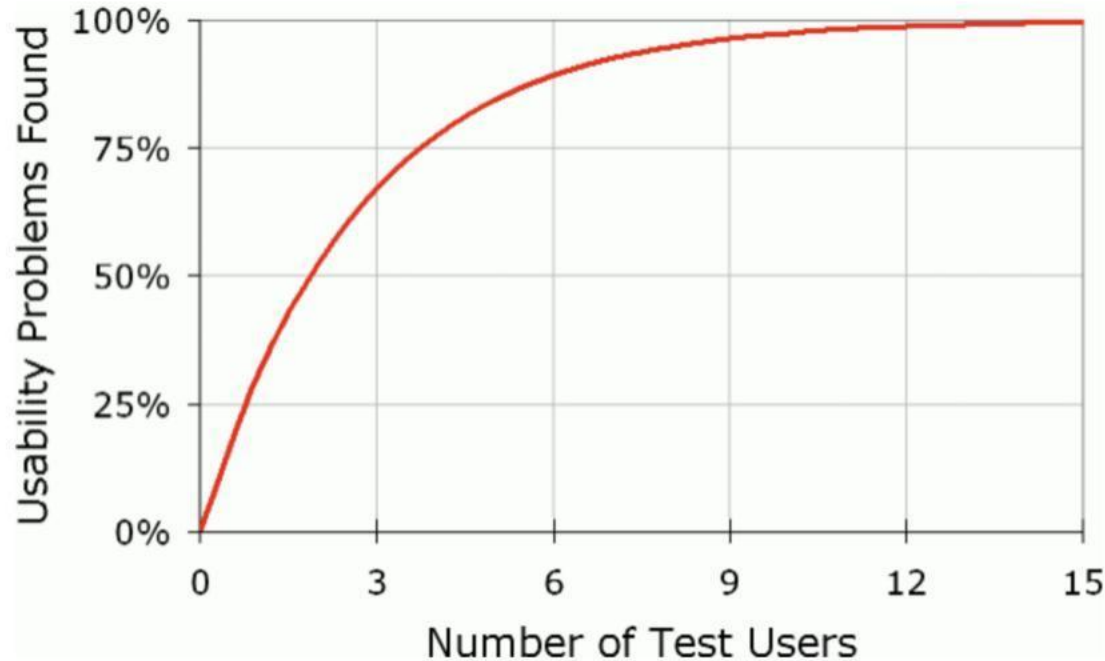


“Dogfooding”

- ❑ **Dogfooding** (from “eating your own dog food”) in the context of software testing means that a company or development team **uses its own product internally** before releasing it to the public.

Aspect	Explanation
Type of Testing	Informal real-world usage testing by actual team members.
Purpose	To catch usability, performance and practical issues before users do.
Stage	Done before full release, often during or internal beta
Testers	Developers, designers, PMs, support staff – ideally everyone in-house.

Revealing Usability concerns



User Study methods

□ Examples:

- Interviews & Surveys
- Shadowing
- Lab study
- Field study
- A/B Testing
- Prototyping

- What are the pros and cons of each ?
- Each answers different questions
- More often than not multiple of these methods are used together

Lab Study

- ❑ Controlled setting with complete control over tasks
- ❑ Common to record the participant and the screen
- ❑ **Metrics** : Task time, correctness, mistakes, opinions.
- ❑ Beware of reactivity effects.
- ❑ Do they use the app as you expected ?

A/B Testing



A/B Testing

- ☐ Compare two variations of a design.
- ☐ Often used for webpages with call to action
- ☐ Randomly show a user design A or B
- ☐ **Does design A or B lead to more sales/ signups/ clicks?**
- ☐ Requires many users to be taken seriously.
- ☐ Is there a *statistically significant difference* ?
- ☐ Doesn't tell you *why* ?

Shadowing

- “fly on the wall”
- Observe people in their own setting... often work.
- Rich qualitative data

- Cons ?
 - Expensive.
 - Time consuming
 - Difficult to get access
 - *Sus*

Prototyping

- ❑ Implementing a full feature/ product is expensive.
- ❑ Prototype helps validate ideas before heavy investment and helps identify user needs, expectations and usability issues.

- ❑ **Mockups/ wireframes** : Visual blueprints of the user interface with basic layout and structure.
- ❑ **Paper prototype** : Hand-drawn sketches representing UI screens or workflows.
- ❑ **Wizard of oz** : Users interact with a system they believe is real, but responses are manually controlled by a human behind the scenes.

Aspects of Usability

- Learnability
- Efficiency
- Satisfaction
- Errors
- Memorability
- LESEM !!



Learnability

- Can users learn to correctly use it ?
 - Can unfamiliar users pick it up ?
 - Can power users learn to become experts with it ?

- How to provide the right guidance to help users?
- They won't read instruction books.
- Visible and labelled buttons help users.

Design Constraints

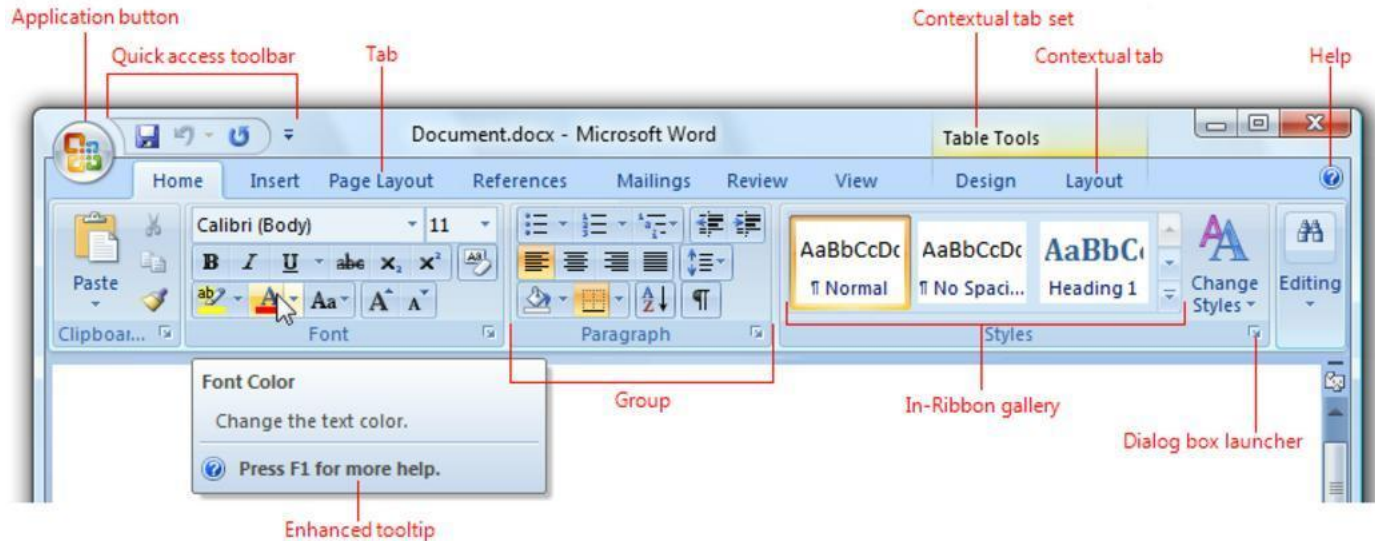
- Don't overwhelm the user.
- Make it easy to find features .
- This is an example of bad design ->



Efficiency

- How fast can users perform the correct interaction?
- How do you design the interfaces for faster interaction ?
- An example of bad design would be if Google pay placed the scan QR Code option deep inside the app instead of making it available in a single tap.

Example of good design – Word 2007



Errors

- Is it unlikely that users will make errors ?
- Is it easy to recover from an error ?
- If errors do come up make them descriptive



Make it hard to make mistakes

- Unambiguous buttons.
- Clear descriptions.
- No surprises.
- Every action should be **reversible**.

Revertible

- ❑ Ever close a tab on accident?
 - Reopen tab
- ❑ Ever close all your tabs ?
 - Reopen all
- ❑ Ever close your web browser ?
 - Recent update made it harder
- ❑ Delete a file ?
 - Recycle bin
- ❑ Changed a file?
 - Undo
- ❑ Clicked the next button ?
 - Previous



Summary

- The **importance of usability** in ensuring software systems are intuitive, efficient, and user-friendly.
- The complete **Usability Engineering Lifecycle**, including:
 - User requirements gathering
 - Task analysis
 - Design & prototyping
 - Usability evaluation methods (formative and summative)
- Methods for **evaluating usability**:
 - Heuristic evaluation
 - Cognitive walkthroughs
 - Usability testing with real users
- **What You Should Take Away**
 - Usability is **not an afterthought** — it must be integrated from the start of the software development lifecycle.
 - A well-designed user interface improves **adoption, efficiency, and satisfaction**, and
 - reduces errors and support costs.
 - Being able to **evaluate usability objectively** makes you a better software engineer and designer.
 - Accessibility is an ethical and practical requirement in all usable systems.



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof. Rajesh Banginwar
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2 – Lecture 7

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Software Quality

Case Study: Heartbleed Vulnerability in OpenSSL

- The Cost of Neglecting Quality Management in Software Projects
- Introduce how lapses in quality processes can lead to pervasive security flaws affecting millions.

Background – What is OpenSSL?

- ❑ OpenSSL is a widely used open-source library implementing SSL/TLS protocols for secure communications. It underpins web servers, mail clients, and many embedded devices.
- ❑ Maintained by a small team of volunteers; updates were irregular and lacked formal release processes at the time.
- ❑ Adopted by major projects (Apache, Nginx, curl), meaning any flaw propagates broadly.

TLS Heartbeat Overview

- ❑ **Protocol Extension (RFC 6520):** TLS Heartbeat lets one peer send a “keep-alive” message—containing an arbitrary payload and a length field—to its counterpart without renegotiating the entire TLS handshake. The receiver is supposed to echo back exactly that payload.
- ❑ **Vulnerability Mechanism:** OpenSSL did **not** validate that the length field matched the actual payload size. A malicious client could ask for, say, 64 KB of data while sending only a few bytes, causing the server to return excess memory contents.
- ❑ **Security Impact:** Memory disclosure at this level can expose private keys, session tokens, and Personally identifiable information (PII) —completely undermining TLS confidentiality guarantees (CVE-2014-0160).
- ❑ **Reference:** <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160>

The Incident – Heartbleed (2014)

- Over two years, the bug remained in code, unnoticed by code reviews and testing.
- **Discovery:** April 7, 2014 — Neel Mehta of Google Security identifies suspicious behaviour in OpenSSL's heartbeat code.'
- **Public Disclosure & Patch Release:** April 7–8, 2014 — OpenSSL team publishes versions 1.0.1g (and later 1.0.0 series) with the fix.
- **Response Window:** Immediately following disclosure, major browsers, OS distributions, and server operators raced to update—yet **17% of HTTPS servers** remained vulnerable for weeks.

Code Sample

```
/* TLS Heartbeat message handler in OpenSSL */
unsigned char *p = /* pointer to received payload */;
unsigned short payload_length = /* length field sent by client */;
unsigned char *buffer = OPENSSL_malloc(payload_length);
if (buffer == NULL) return;

/* Vulnerability: copying payload_length bytes from p into buffer
   without verifying if the actual received payload length matches payload_length */
memcpy(buffer, p, payload_length);

/* The server then responds with buffer contents, leaking extra memory */
```

What quality processes failed here?

Quality Management Failures

- ❑ **Lack of Formal Code Review:** Patches were accepted without systematic peer review or documented approval workflows.
- ❑ **Insufficient Testing:** No negative tests for out-of-bounds reads of heartbeat messages; missing fuzzing or boundary-case tests.
- ❑ **Unstructured Release Process:** No formal quality gates or regression test suites enforced before release.
- ❑ **Poor Documentation:** Limited design specs and no comprehensive test plans made maintenance harder.

What Should Have Been Done?

- ❑ **Defined Quality Framework:** Establish a release process with mandatory code reviews, test coverage thresholds, and approval steps.
- ❑ **Automated Testing:** Integrate unit tests, boundary-value tests, and fuzz testing in CI pipelines.
- ❑ **Documentation & Traceability:** Maintain clear design documents, change logs, and test plans to track requirements and coverage.
- ❑ **Periodic Audits:** Conduct security audits and peer reviews on critical modules



- ❑ Software systems are complex and evolve, hence there is a need for continuous assessment and evaluation of quality
- ❑ Bad quality of software could have serious repercussions on life if we are looking at life-critical applications.
- ❑ Can definitely lead to dissatisfied customers
- ❑ Software quality enhances long term profitability of products

Software Quality – Different perspectives

□ **Transcendent**

- Exceeded normal expectations.

□ **User-Based**

- Fitness for use

□ **Manufacturing based**

- Conformance to specs.

□ **Product-based**

- Based on attributes of the software

□ **Balancing time, cost and profits**

- Value-Based

Product Operation Perspective

- ❑ **Correctness:** Does it do what I want?
- ❑ **Reliability:** Is it always accurate?
- ❑ **Efficiency:** Does it run as well as it can?
- ❑ **Integrity:** Is it secure?
- ❑ **Usability:** Can I use it?
- ❑ **Functionality:** Does it have necessary features?
- ❑ **Availability:** Will the product always run when needed?

Software Product quality perspective

Overall Environment Perspective

- ❑ **Responsiveness:** Can I quickly respond to change?
- ❑ **Predictability:** Can I always predict the progress?
- ❑ **Productivity:** Will things be done efficiently?
- ❑ **People:** Will the customers be satisfied?

Will the employees be gainfully engaged?

Software Product quality perspective

Product Revision Perspective

- ❑ **Maintainability:** Can I fix it?
- ❑ **Testability:** Can I test it?
- ❑ **Flexibility:** Can I change it?

Product Transition Perspective

- ❑ **Portability:** Can it be used on another machine?
- ❑ **Reusability:** Can I reuse some/all of the software?
- ❑ **Interoperability:** Can I interface it with another system?

Approaches to look at quality

Quality of a software product can be visualized as “Quality of the Product” vs “Quality of the Process”

Quality actions could include checking whether it conforms to certain norms

Verification, Validation and Audits

Improvement in quality could be achieved by improving the product or process

Quality Attributes (FLURPS+)

- ❑ **Functionality:** features of system
- ❑ **Localization:** Localizable to local language
- ❑ **Usability:** Intuitive, documentation
- ❑ **Reliability:** Frequency of failure in intended time
- ❑ **Performance:** Speed, throughput, resource consumption
- ❑ **Supportability:** Serviceability, maintainability
- ❑ The + could include Extensibility and so on

Software quality - Terminology

- **Attribute:** measurable physical or abstract property of an entity
- **Measure:** Quantitative indication of extent, amount, dimension, capacity or size
Eg: Number of errors
- **Metric:** Quantitative measure of degree to which a system possesses a given attribute
Calculation between two measures
Eg: number of errors found per person hours

Why measure?

- **Feedback:** quantifies some of the attribute to help improve the product
- **Diagnostics:** helps identify issues towards quality
- Supports evaluating and establishing productivity
- **Forecasting:** to predict future need and anticipate maintenance
- Supports estimating, budgeting, costing and scheduling

- ❑ **Quantitative:** Metrics should be quantitative and expressible in values
- ❑ **Understandable:** Metric computation should be defined and easily understood
- ❑ **Applicability:** Should be applicable at all stages of software development
- ❑ **Repeatable:** Metrics are consistent and same when measured again
- ❑ **Economical:** Computation of metric should be economical
- ❑ **Language Independent:** Metrics should not depend on programming language

Examples of Measures

□ **Correctness**

Defects/KLoC, Failures/Hours of operation

□ **Maintainability**

Mean time to change, Cost to correct

□ **Integrity**

Fault tolerance, security and threats

□ **Usability**

Training time, skill level, productivity

- Cost of quality (COQ) is defined as a methodology that allows an organization to determine the extent to which its resources are used for activities that **prevent poor quality**, that appraise the quality of the organization's products or services, and that result from internal and external failures.
- Having such information allows an organization to determine the potential savings to be gained by implementing process improvements.
- Cost of Good Quality (COGQ) and Cost of Poor Quality (COPQ)
 - COGQ consists of the cost of quality conformance, including any associated costs with both appraisal and prevention, whereas COPQ involves all the nonconformance costs that are both internal and external to the company.

Cost of software quality

- Measure of quantifying and calculating business value of quality activities
- Costs incurred through meeting and not meeting customer quality
- Isolates the state of quality in the company and helps eliminate waste due to poor quality

Cost of Good Quality	Cost of Bad Quality
Prevention costs: Investments to prevent/avoid quality problems E.g.: Training staff, designing quality into the product, quality planning	Internal failure costs: Fixing bugs before product reaches customers E.g.: rework, scrap, delays
Appraisal costs: costs to determine degree of conformance to requirements and quality standards E.g.: Inspections, testing, audits to catch defects before release	External failure costs: costs associated with defects found after customer receives product E.g.: Support Calls, Patches
Management Control costs: costs to prevent or reduce failures in management functions E.g.: contract reviews, gating/release criteria	Technical debt: cost of fixing a problem, <u>which left unfixed, puts the business at risk</u> E.g.: Structural problems, Increased Complexity
	Management failures: costs incurred by personnel due to poor quality software Eg: Unplanned costs, customer damages

Software Quality Assurance: methods to monitor software engineering process to ensure quality

- Involves planning (setting up of goals, commitments, activities, measurements and verifications)
- Encompasses
 - Entire software development processes and activities
 - Planning oversight, record keeping, analysis and reporting
 - Auditing designated software work to verify compliance
 - Ensuring deviations from documented procedure are recorded and noncompliance is reported

Contents of the SQA plan

- Quality cannot be plugged in and needs to happen throughout the lifecycle

- Addresses the following
 - Responsibility Management
 - Document Management and Control
 - Requirements Scope
 - Design Control
 - Development Control and Rigor
 - Testing and Quality Assurance
 - Risks and Mitigation
 - Quality Audits
 - Defect Management
 - Training Requirements



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof. Rajesh Banginwar
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 2 – Lecture 7

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Technical Debt

Technical Debt



A better analogy?: Pollution



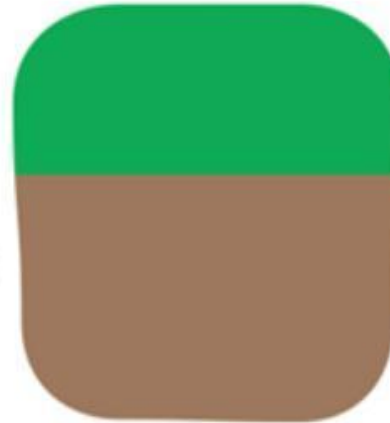
AQI Basics for Ozone and Particle Pollution			
Daily AQI Color	Levels of Concern	Values of Index	Description of Air Quality
Green	Good	0 to 50	Air quality is satisfactory, and air pollution poses little or no risk.
Yellow	Moderate	51 to 100	Air quality is acceptable. However, there may be a risk for some people, particularly those who are unusually sensitive to air pollution.
Orange	Unhealthy for Sensitive Groups	101 to 150	Members of sensitive groups may experience health effects. The general public is less likely to be affected.
Red	Unhealthy	151 to 200	Some members of the general public may experience health effects; members of sensitive groups may experience more serious health effects.
Purple	Very Unhealthy	201 to 300	Health alert: The risk of health effects is increased for everyone.
Maroon	Hazardous	301 and higher	Health warning of emergency conditions: everyone is more likely to be affected.

<https://www.airnow.gov/aqi/aqi-basics>

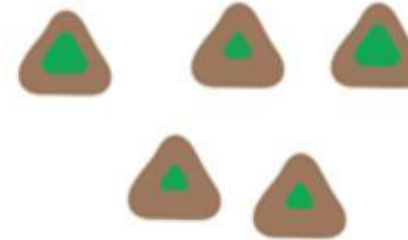
Technical Debt

Any software system has
a certain amount of
essential complexity
required to do its job...

... but most systems
contain *cruft* that makes it
harder to understand.

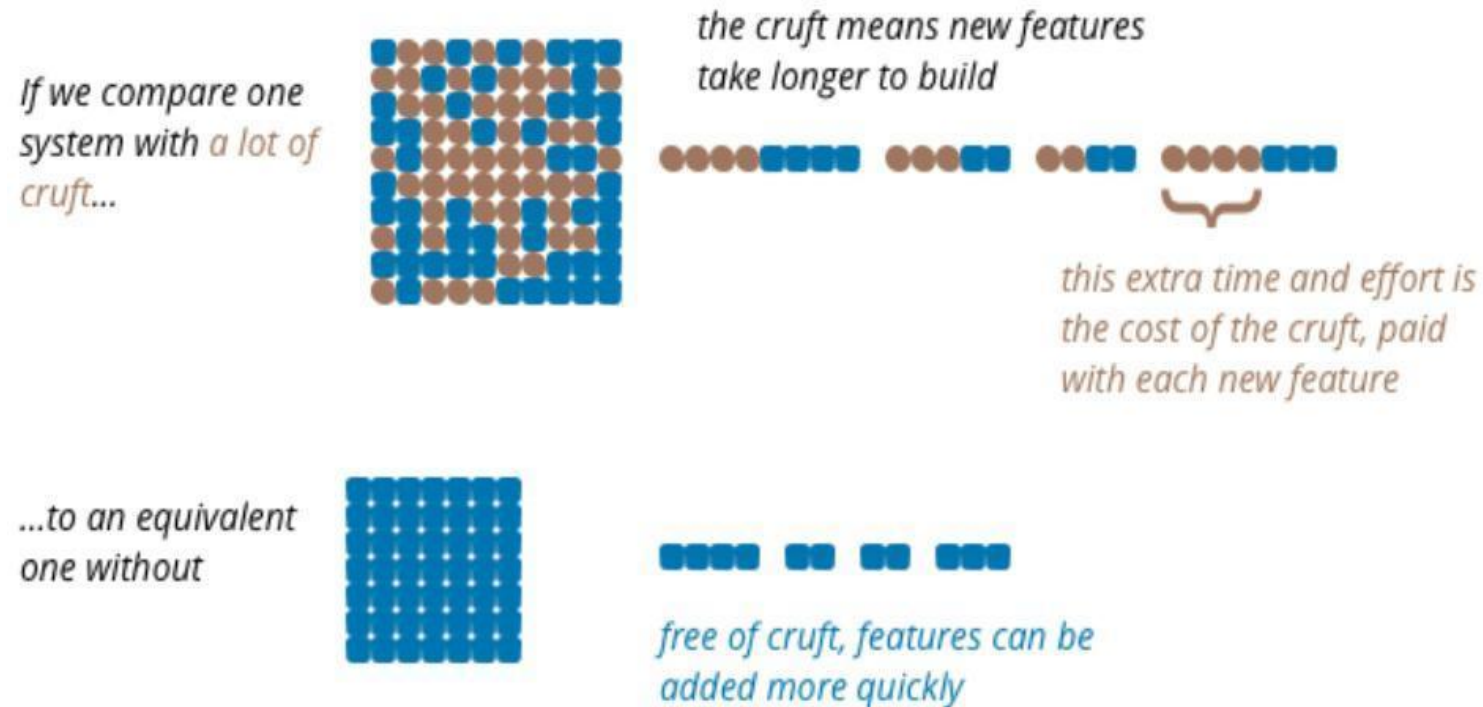


Cruft causes changes
to take *more effort*



The technical debt metaphor treats the
cruft as a debt, whose interest payments
are the extra effort these changes require.

Internal quality makes it easier to add features



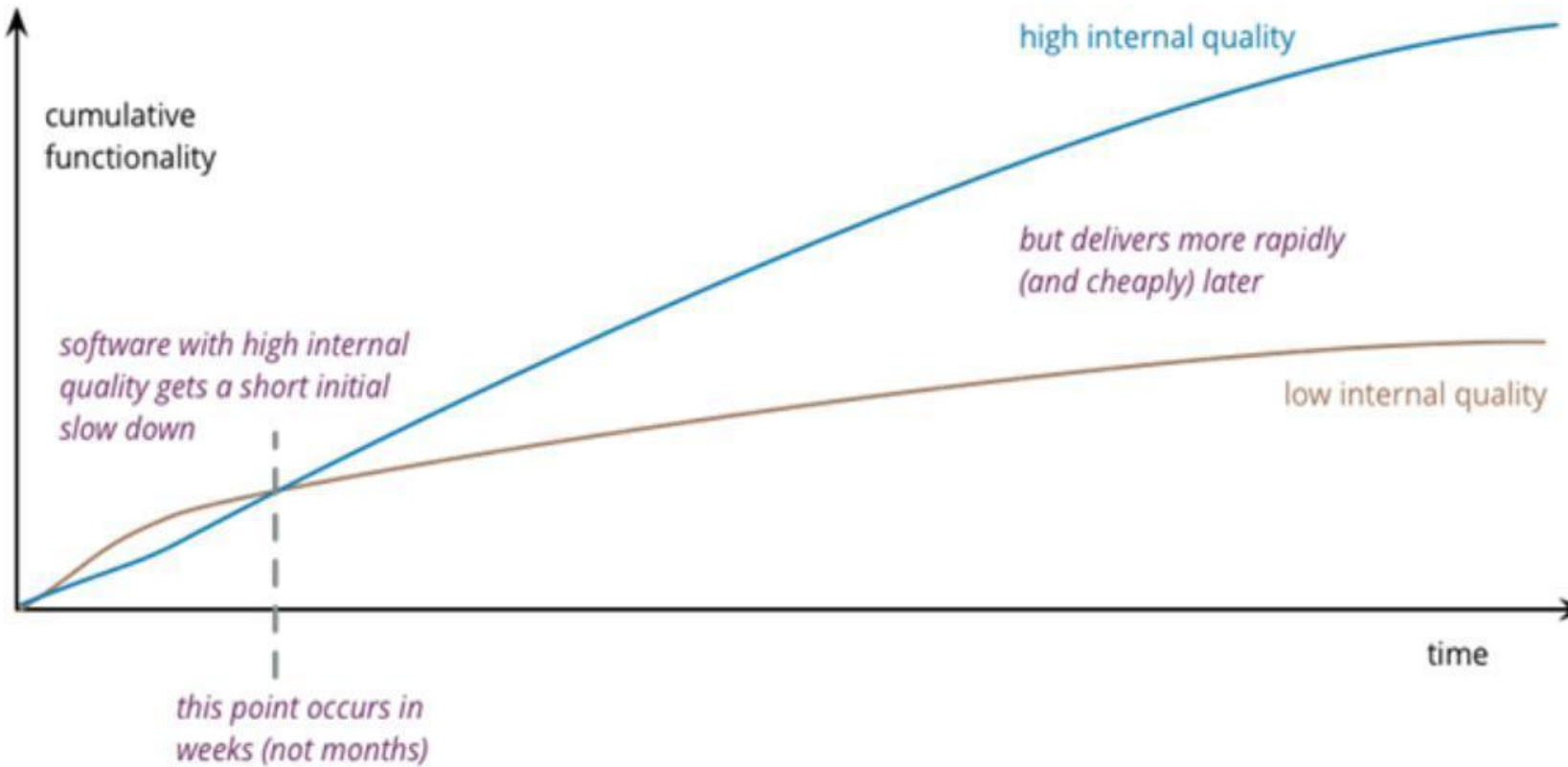
Examples of Technical Debt

Technical Debt != Bad Internal Quality

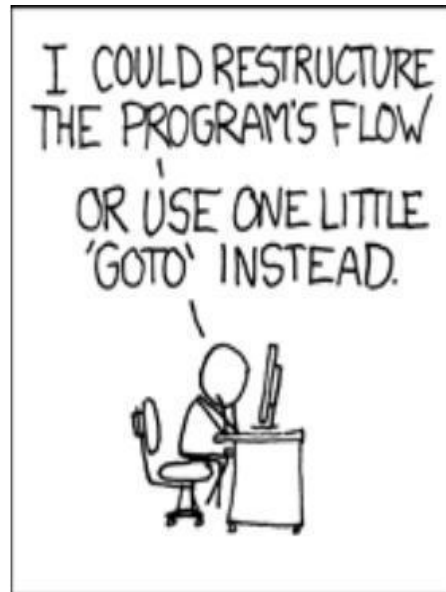
*“In software-intensive systems, technical debt consists of **design or implementation constructs** that are expedient in the **short term** but that set up a technical context that can make a **future change more costly or impossible**. ”*

*“Technical debt is a contingent liability whose impact is **limited to internal system qualities** – primarily, but not only, **maintainability** and **evolvability**.”*

High Internal Quality is an Investment



What actions cause technical debt?



What actions cause technical debt?

- ❑ Tightly-coupled components
- ❑ Poorly-specified requirements
- ❑ Business pressure
- ❑ Lack of process
- ❑ Lack of documentation
- ❑ Lack of automated testing
- ❑ Lack of knowledge
- ❑ Lack of ownership
- ❑ Delayed refactoring
- ❑ Multiple, long-lived development branches

What actions cause technical debt?

**EVERYONE
CREATES
TECHNICAL DEBT**



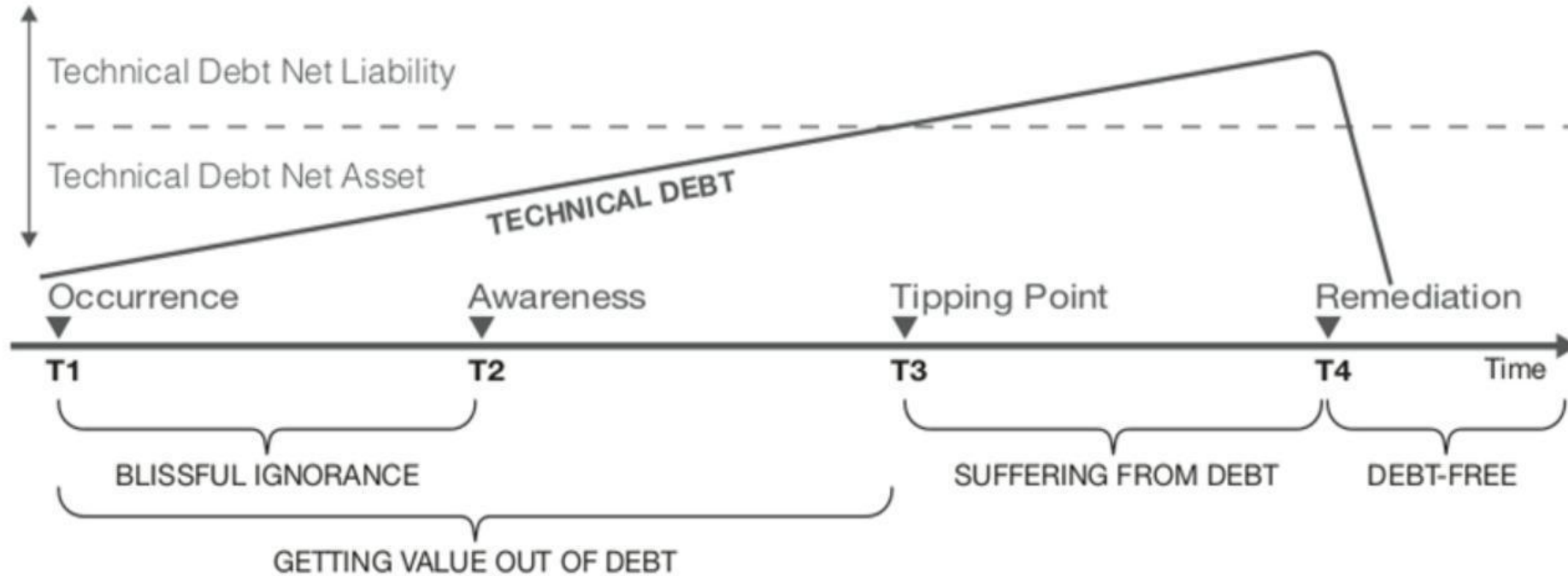
Bad: Too much technical debt

- ❑ Bad code can be demoralizing
- ❑ Conversations with the client become awkward
- ❑ Team infighting
- ❑ Turnover and attrition
- ❑ Development speed



How to manage Technical Debt?

When should we reduce technical debt?



Managing Technical Debt

Organizations needs to address the following challenges continuously:

- Recognizing technical debt
- Making technical debt visible
- Deciding when and how to resolve debt
- Living with technical debt

Not all Technical Debt is same

	Reckless	Prudent
Deliberate	<i>"We don't have time for design"</i>	<i>"We must ship now and deal with consequences (later)"</i>
Inadvertent	<i>"What's layering?"</i>	<i>"Now we know how we should have done it"</i>

How can we avoid (inadvertent) technical debt?

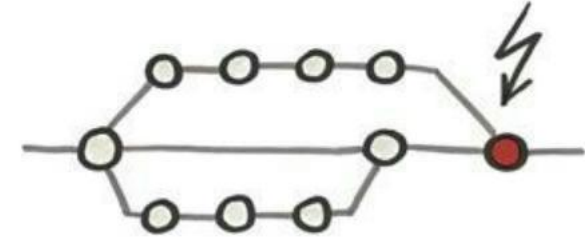
Common Anti-Patterns

- Not having a QA process! Or no-one follows it



Common Anti-Patterns

- ❑ Not having a QA process! Or no-one follows it
- ❑ Bad version control practices
 - Everyone commits to the main branch
 - Long-lived feature branches
 - Huge PRs



	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJ5OKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAHAHAHAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

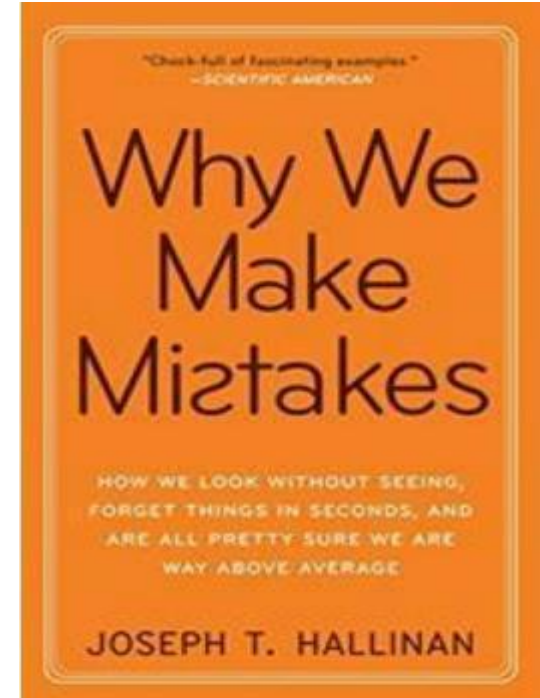
Common Anti-Patterns

- ❑ Not having a QA process! Or no-one follows it
- ❑ Bad version control practices
- ❑ Slow and encumbering QA processes
 - changes take forever to get merged
 - time could be better spent on new features



Common Anti-Patterns

- ❑ Not having a QA process! Or no-one follows it
- ❑ Bad version control practices
- ❑ Slow and encumbering QA processes
- ❑ Reliance on repetitive manual labor
 - focused on superficial problems rather than structural ones
 - results may vary (e.g., manual testing)
 - mistakes will happen!



Case Study: Knight Capital

Date: August 1, 2012

Company: Knight Capital Group

Industry: Financial services (high-frequency trading)

Event: A software deployment error caused Knight Capital to lose **\$460 million in 45 minutes**, leading to near-bankruptcy.

Deployment Failure:

The new code was **only deployed to 7 out of 8 servers**.

One server continued running **outdated, test-mode code** from 2003 which sent **erroneous stock orders**.



In layman's terms, Knight Capital Group realized a \$460 million loss in 45-minutes. Remember, Knight only has \$365 million in cash and equivalents. **In 45-minutes Knight went from being the largest trader in US equities and a major market maker in the NYSE and NASDAQ to bankrupt.**

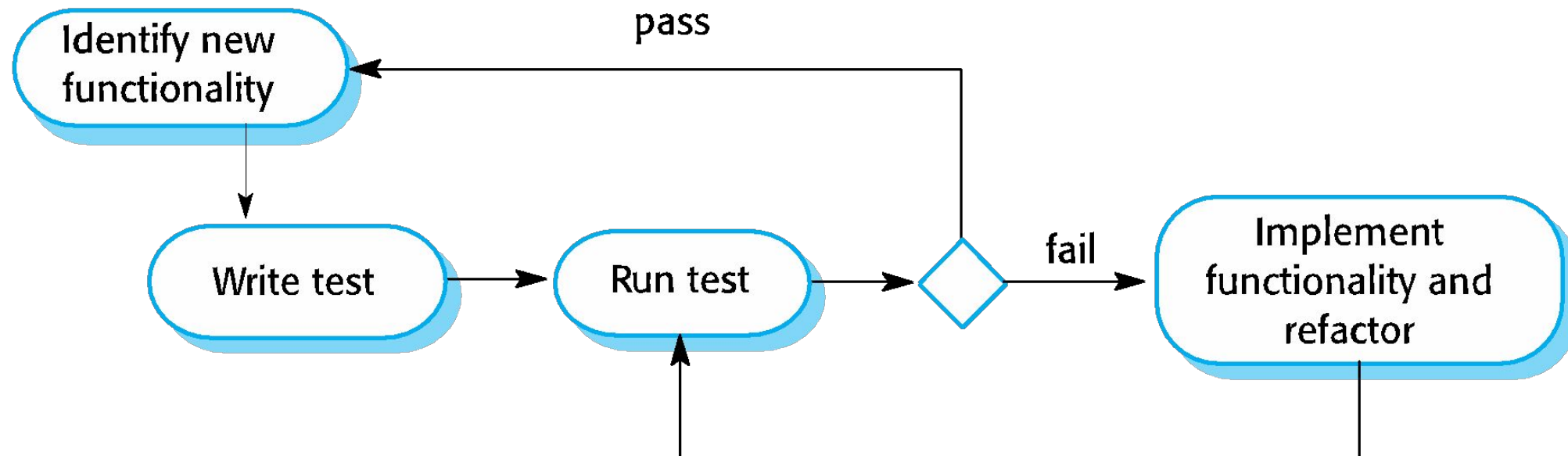
- **Scenario:** Imagine your development team is building the Campus Event Management app. Due to tight deadlines, several shortcuts were taken.
- List **three examples** of technical debt that might occur during development of the app.
 - For each example, specify:
 - What is the debt? (e.g., "No automated tests")
 - Cause of debt (e.g., "Skipped testing to meet deadline")
 - Potential impact (e.g., "Harder to debug and fix bugs later")
 - From your list, select the **most urgent technical debt** to address.
 - How to mitigate this debt (e.g., "Add test automation framework")
 - When should it be remediated? (e.g., "Before next sprint release")

Test-driven Development

Test-driven Development

- Test-driven development (TDD) is an approach to program development in which you inter-leave testing and code development.
- Tests are written before code and ‘passing’ the tests is the critical driver of development.
- You develop code incrementally, along with a test for that increment. You don’t move on to the next increment until the code that you have developed passes its test.
- TDD was introduced as part of agile methods such as Extreme Programming. However, it can also be used in plan-driven development processes.

Test-driven Development



TDD process activities

- ❑ Start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- ❑ Write a test for this functionality and implement this as an automated test.
- ❑ Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- ❑ Implement the functionality and re-run the test.
- ❑ Once all tests run successfully, you move on to implementing the next chunk of functionality.

Benefits of test-driven development

- Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
 - A regression test suite is developed incrementally as a program is developed.
- Simplified debugging
 - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.
- System documentation
 - The tests themselves are a form of documentation that describe what the code should be doing.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.